
***Risk Management in Stock Markets and Portfolio
Optimization Using Physics-Inspired Machine Learning.***

Hridayesh Chettri

B.Sc. Physics (Honours)

Roll no. 167, Semester VI

Supervisor: Prof. (Dr.) Shibaji Banerjee (H.O.D.)

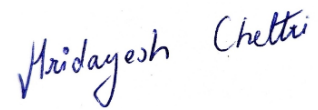
Post Graduate and Research Department of Physics

St. Xavire's College (Autonomous), Kolkata.



Declaration

I hereby declare that I have conducted this dissertation paper with the utmost care and professionalism. I confirm that all sources of information, including published and unpublished materials, have been appropriately identified, and that no part of this paper has been written using unacknowledged materials.



-Hridayesh Chettri

Acknowledgement

I would like to convey my sincere gratitude and admiration to my supervisor, Prof.(Dr.) Shibaji Banerjee, for his unwavering support and guidance. Without him this project would not have been possible.

I would also like to express my gratitude to my parents for always believing in me and being supportive of my decisions no matter the circumstances. I am also very grateful towards all my friends for providing me ideas and useful insights.

Contents

Introduction	1
1 The Basics	2
1.1 Machine Learning	2
1.2 Supervised Learning	5
1.3 Python and its machine learning module: Scikit-learn	6
1.3.1 Python	6
1.3.2 Scikit-Learn	6
1.4 Stock, Asset and Portfolio Optimization	7
2 Machine Learning and Stock Market Analysis	9
2.1 Data Set Description	9
2.2 Entropy	10
2.2.1 Introduction	10
2.2.2 Entropy in Physics	10
2.2.3 Entropy in Information Theory	12
2.2.4 Entropy in Finance	12
2.2.5 Entropy in our program	15
2.3 Brownian Motion	16
2.3.1 Introduction	16
2.3.2 Brownian Motion in Physics	16
2.3.3 Brownian Motion in Finance	18
2.3.4 Brownian motion in our program	19
2.4 Volatility	20
2.4.1 Introduction	20
2.4.2 Volatility in Physics and Mathematics	20
2.4.3 Volatility in finance	21
2.4.4 Volatility in our program	23

2.5	Simple Harmonic Motion (SHM)	23
2.5.1	Introduction	23
2.5.2	Simple Harmonic Motion in Physics	24
2.5.3	Simple Harmonic Motion in Finance	25
2.5.4	Simple Harmonic Motion in our program	27
3	The Program	28
3.1	Python Libraries Imported	28
3.2	train_test_split	30
3.3	StandardScaler	31
3.4	RandomForestRegressor	32
3.5	Pandas	34
3.6	cache_data.py	36
3.6.1	Importing Modules	36
3.6.2	Passing the API and the base URL	36
3.6.3	Defining Function fetch_stock_data	37
3.6.4	Defining function cache_stock_data	39
3.7	dissertation.py	41
3.7.1	Importing relevant modules	42
3.7.2	Loading the Cached Data	42
3.7.3	Combining the Dataframes	44
3.7.4	Calculating Daily Log returns	45
3.7.5	Defining Physics-inspired features	46
3.7.6	Defining Target Variables	49
3.7.7	Train-Test Split	50
3.7.8	Data Scaling	51
3.7.9	Model Training (Random Forest)	52
3.7.10	Predictions	54
3.7.11	Visualization:-	54
4	Output, Evaluation of Performance of the Model and Deductions	56
4.1	Evaluation of Regression model	56
4.1.1	Mean Squared Error (MSE)	57
4.1.2	Mean Absolute Error (MAE)	57
4.1.3	R^2 (R-squared)	57
4.2	Output	58
4.2.1	Output for our base program	58

4.2.2	Modified Program/Inputs	59
Conclusion		67

Introduction

The Barter system is considered the oldest form of trade that humans used to practice long before money was invented. People used to trade their goods and services (i.e. their assets) for other goods and services in return. This was the primary and only method of trade then. We have come a long way since then.

Today, we have what are called **stocks**, built on a bizarre notion that would have been inconceivable to the ancient people. It is a form of asset that can be legally owned and represents a fraction of a company or industry. It is something tangible, as the company has real assets. However, it is traded digitally and its value is not fixed, but dynamic, depending on the market. Their values fluctuate due to market sentiment, news, or speculation. Thus, while **volatility in the stock market introduces significant risk, it also offers the potential for remarkable returns**. This is what drives people to optimize their portfolio, minimizing risk and maximizing returns.

This project deals with **using Physics-inspired machine learning algorithm to optimize portfolio**. We try to see what role does concepts of Physics such as **Entropy, Brownian Motion, Volatility and Simple harmonic motion** play in the Stock market. To achieve our goal, we train **RandomForestRegressor**, a machine learning model of Scikit-learn (a library in Python), with historical stock data and the data calculated from functions derived using these concepts of Physics.

The output we get shows us how well our predictive model can work in order to predict market trends using Physics integrated with Machine Learning. From our output, we can deduce what the best scheme for investing and optimizing our assets would be in order to gain highest returns with the lowest risk.

Chapter 1

The Basics

Everything in this universe has a beginning. We humans have always endeavored to understand and apply our understanding on everything around us. To begin the process of understanding or applying something, we must start from the very basics. It is the basics that helps us uncover the beautiful mysteries of life. It is the basics that help us build mind boggling systems and applications. Without the basics we will have nothing fruitful. Therefore, we begin our project with the basics.

1.1 Machine Learning

We live in a world driven by data. Data are everywhere and are the foundations for newer technologies and changes. Obtaining, storing, analysing and using data is the key to modern revolution. In this world, data is power and being able to use data gives one the power to conquer the world. One of the most influential field that has taken over this realm of data is Machine Learning. It is truly the pinnacle of data analysis in the present times.

Machine learning is a branch of artificial intelligence that focuses on creating algorithms capable of improving their performance over time based on experience and data. It allows computers to analyse information, recognize patterns, and make informed decisions without requiring explicit programming for every task. This technology leverages various self-learning methods to extract insights from data and predict future outcomes. Unlike traditional programming, which follows fixed rules, machine learning systems dynami-



Figure 1.1: Pictorial representation of Machine Learning[1]

cally adapt by detecting trends, making forecasts, and refining their responses based on prior observations.

At its core, machine learning involves feeding the computers with massive amounts of data. The learning method of these algorithms can be broadly classified into three categories. They are:-

- **Supervised Learning:** In this approach, algorithms are trained using labelled datasets to identify patterns and make predictions. The model establishes a relationship between input features and corresponding output labels during training. These datasets consist of paired inputs and outputs, allowing the model to associate inputs with correct results. Common supervised learning techniques include regression, which predicts continuous values, and classification, which categorizes data into predefined groups.
- **Unsupervised Learning:** This method involves training a model using unlabelled datasets, allowing it to identify patterns and group data based on similarities without explicit instructions. The goal is to uncover hidden structures within the data without human intervention. Common techniques include clustering, which groups similar data points, and dimensionality reduction, which simplifies datasets by minimizing the number of features while preserving essential patterns.
- **Reinforced Learning:** Agent or system learns to make decisions by interaction with the environment where it is applied. It is rewarded or punished (with points) for its actions, and its goal is to maximize its points earned. Training a model to make sequences of decisions by rewarding desired behaviours and/or punishing undesired ones. This approach has been particularly effective in fields like robotics and game

playing, where an agent learns optimal strategies through trial and error.

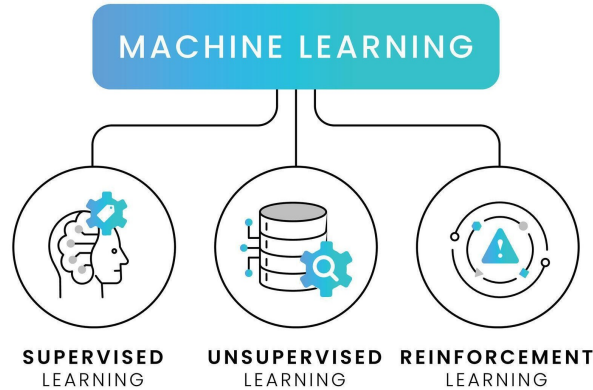


Figure 1.2: Representation of different types of Machine Learning[2]

However, it must be noted that machine learning is not absolute. Quality and quantity of data is key, as well as on proper model selection and tuning. Overfitting is one of the many problems that it might face. To address this, techniques such as cross-validation, regularization, and ensemble methods are often employed.

Nonetheless, machine learning is highly useful in every field. In health-care, it's used for diagnosing diseases from medical images and predicting correct course of action in treatment. In finance it is used in portfolio optimization, fraud detection and algorithm trading. In natural language processing, machine learning has applications like language translation, sentiment analysis, and chat-bots. Also, there is the ever increasing application in deep learning that have led to breakthroughs in computer vision and speech recognition.

Machine learning has not only helped in decision making but it has also helped us to figure out the relationships between complex data-sets and its applications are countless. Its continuous evolution, driven by both research and practical applications, makes it a cornerstone technology in the modern era of data-driven decision-making.

1.2 Supervised Learning

Supervised learning is a key area of machine learning that involves training models on labelled datasets. In this approach, each data point in the training set is associated with a corresponding output value or label. The objective is to enable the model to recognize patterns and establish relationships between inputs and outputs, allowing it to make accurate predictions on new, unseen data. This learning method mirrors human learning, in which individuals observe inputs, analyse outcomes, and form associations. It is often compared to a teacher guiding a student, which is why it is termed "supervised" learning.[3]

We assume that there is a function $\mathbf{f}$ such that for a given input $\mathbf{x}$ there is an output y that can be approximated by $y \approx f(x)$. The goal of supervised learning is to find $f(x)$ that minimizes the difference between the predicted outputs and the actual labels.

Supervised learning can further be divided in to two primary types:-

- **Classification:** It is a form of supervised learning in which input data is assigned to predefined categories or labels. The model identifies patterns from labelled training data and applies them to classify new, unseen data accurately. This technique is widely used in applications such as spam detection, medical diagnosis, and sentiment analysis. Some commonly used classification algorithms include logistic regression, decision trees, support vector machines (SVM), and neural networks.
- **Regression:** It is a supervised learning method aimed at determining a continuous numerical outcome from given input parameters or factors. Instead of sorting data into categories, regression analyses numerical relationships to predict future values. It is frequently used in areas such as real estate price forecasting, financial market analysis, and climate prediction. Popular regression approaches include linear regression, polynomial regression, support vector regression (SVR), and decision tree regression.

Supervised learning is very powerful as can produce models that make accurate predictions on unseen data. This capability is critical in real-world

applications. For example, in healthcare, supervised learning models help diagnose diseases by analyzing a patient's data. In finance, these models can forecast stock prices or detect fraudulent transactions. **We will also be using supervised learning for our project** as we have to predict the stock prices for which we will be using **historical labelled stock prices**.

1.3 Python and its machine learning module: Scikit-learn

1.3.1 Python

The programming language that we will be using for our project is **Python**. It is a high-level programming language as it uses natural language as codes. It was created by Guido van Rossum in 1991. It is the most famously used programming language for a wide range of applications.

One of the most important feature of python is its easy-to-understand syntax, which makes it accessible for both beginners and expert developers. It supports many kinds of programming techniques such as object-oriented, procedural, and functional programming. Python comes with a vast standard library, reducing the need for additional code when handling file operations, networking, or data processing.

Python is popularly used in machine learning and data science because of its powerful machine learning and data handling libraries like NumPy, Pandas, Scikit-Learn, TensorFlow, and PyTorch. Moreover, Python's strong community support ensures constant updates, extensive documentation, and a wealth of third-party libraries. Its ability to integrate seamlessly with C, C++, Java, and other languages makes it highly flexible. Due to its efficiency and broad applicability, Python continues to be a dominant force in programming across industries.

1.3.2 Scikit-Learn

Scikit-learn is a free Python library for machine learning that works with both supervised and unsupervised methods. It offers tools to train models, clean and prepare data, choose the right model, check a model's performance,

and more. Built with SciPy, NumPy, and Matplotlib, it runs efficiently and is easy to use. Thanks to its simplicity, flexibility, and speed, Scikit-learn is a very famous library among machine learning professionals and researchers.[4]

The scikit-learn project started as scikits.learn, a Google Summer of Code project by French data scientist David Cournapeau. The name of the project stems from the notion that it is a "SciKit" (SciPy Toolkit), a separately developed and distributed third-party extension to SciPy. The original code-base was later rewritten by other developers. In 2010, contributors Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort and Vincent Michel, from the French Institute for Research in Computer Science and Automation in Saclay, France, took leadership of the project and released the first public version of the library on February 1, 2010. [5]

Scikit-learn allows you to run powerful machine learning models using only a few lines of Python code. Thanks to its efficient design, it can handle large amounts of data, making it practical for everyday industry needs. It also works smoothly with other popular Python libraries like Pandas, TensorFlow, Keras, and XGBoost, which makes it an essential part of the modern data science toolkit.

Scikit-learn being so good and easy for machine learning program is a great start for beginners like us.

1.4 Stock, Asset and Portfolio Optimization

Stocks/Shares represents ownership in a company and signifies a claim on its assets and earnings. Owning one share means we have a small piece of the company, with our piece being a fraction based on the total number of shares available. Typically, if we own a share, we have the right to receive a part of the company's profits, get a portion of the money if the company's assets are sold, or even vote on company decisions. Companies issue stocks to raise capital for their company, and investors buy them in hopes of earning returns through price appreciation and dividends. Stocks are bought and sold on marketplaces, such as the New York Stock Exchange (NYSE) and Nasdaq.

Asset is anything of value that an individual, business, or government owns and can use to generate economic benefits. It is any resource owned

or controlled by a business or an economic entity. Something which has a positive economic value. Assets can be tangible, like cash, real estate, machinery, and inventory, or intangible, like patents, trademarks, and brand reputation. They play a crucial role in wealth creation, business operations, and financial stability, as they contribute to income generation and overall net worth.

Portfolio Optimization is the calculated process of selecting the best assets and stocks in order to maximize net wealth and returns, and minimize risk. It is one of the most fundamental aspect in financial management used by investors, portfolio managers, and financial institutions to construct investment portfolios that achieve the best possible risk-return trade-off.

The return of a portfolio is the sum of the returns of individual assets while risk is the variation/fluctuation in this return. Our goal is calculating and predicting the best scenario of high returns and low risk. Also, according to the Modern Portfolio Theory, assets should be chosen based on their interaction within a portfolio rather than individually. This calculation and prediction is exactly what we are trying to achieve in our program/project. Portfolio optimization is very crucial in financial decision-making, therefore, we are trying to incorporate Physics-Inspired Machine learning algorithm for Portfolio Optimization and see how well physics can help in predicting financial markets.

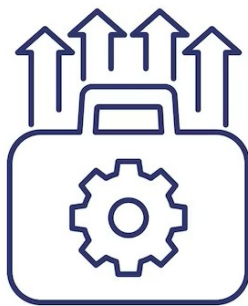


Figure 1.3: Portfolio Optimization[6]

Chapter 2

Machine Learning and Stock Market Analysis

In the earlier sections, we learned the basics behind Finance, Stock Market and what it means. Now we move towards understanding how we can make use of machine learning in this field of the stock market. Let us first begin by talking about the data set.

2.1 Data Set Description

For our data set, we require historical stock data. For this purpose, we have used **Tiingo**[7]. It is a reliable provider that can provide historical stock data via an API key. An API key is like a password that helps different software programs talk to each other and share information safely. In our case, the Tiingo API serves as an intermediary between our data analysis scripts and Tiingo's extensive database of historical stock prices. We use our unique API key to request data for popular stocks such as AAPL (Apple), MSFT (Microsoft), GOOG (Google), and AMZN (Amazon). We request data over a date range (for example, from June 1, 2023, to August 10, 2025). This ensures that we have enough historical records for a meaningful analysis. The API returns the data in a structured JSON format (in the form of program codes), which we then convert into a table using Python's Panda library.

Tiingo is a reliable source because it retrieves data directly from major stock exchanges. The data then undergoes validation processes. Its reliability can be verified by cross-checking the imported data with other reputable sources, such as Yahoo Finance, Alpha Advantage or official exchange reports. Also, to maintain a fixed set of data and to improve the efficiency of data retrieval process, we use a caching method(saving method). After the first download, we save the data locally in text files. This helps us to deal with the limited rate of usage of the data re-transfer process by minimizing repeated API requests. It also ensures that we have consistent and offline access to the dataset.

Although alternatives like Yahoo Finance, Alpha Vantage, and so on are available, we have chosen Tiingo for its ease of use, generous limits of free access, and overall data reliability. As can be observed, our focus is mainly on historical end-of-day data. We have a reason for doing this. It allows us to back-test machine learning models and perform risk analysis. This method is highly suited for academic purposes as it allows for an automated data retrieval, which can then be saved to obtain a reliable and consistent set of data.

2.2 Entropy

2.2.1 Introduction

The Universe around us is filled with mysteries so profound that we sometimes feel powerless. We have come up with many concepts to understand the world in and around us. Our constant quest for the "Theory of Everything" never ceases. One such explanation we have for our universe is characterized by a physical feature called Entropy.

2.2.2 Entropy in Physics

Entropy is one of the most important concepts of physics that governs everything, from atomic interactions to getting life-sustaining energy on Earth from the Sun. In physics, entropy is a measure of the **disorderedness or**

randomness of a system, which indicates the availability of energy not available for doing work. Entropy can easily be understood as disorder. According to the second law of thermodynamics, energy spreads over time. According to this law, entropy measures how energy is distributed among the microscopic states of a system. It states that entropy tends to increase over time, leading to disorderedness. The tendency of energy to spread over can be defined as entropy. While studying thermal physics, we make the fundamental assumption that any closed system is likely to be in any quantum state that is accessible to it. All accessible quantum states are equally probable. [8]

Let us consider an example to understand entropy better. Consider a vessel containing a partition. One side is filled with gas while the other side is a vacuum. When the partition is removed the gas molecules immediately spread throughout the entire vessel. In the initial stage the gas is confined to one side and hence is in a higher entropy state. As soon as the partition is removed, gas molecules spread evenly throughout the entire vessel. In this condition the disorder increases, meaning the number of possible microstates Ω , has increased. Now, we observe that the gas molecules does not automatically return to it's initial state unless some external work is done. This is the 2nd Law of Thermodynamics, which states that entropy of an isolated system tends to increase over time. [9]

There are many different expressions for Entropy in physics. In statistical Mechanics and for the above example we can use the Boltzmann Entropy Formula given by [8]

$$S = k_B \ln \Omega \quad (2.1)$$

where,

S = Entropy,

$k_B = 1.38 \times 10^{-23}$ J/K,

Ω = Number of microstates corresponding to a given macrostate.

Thus, we see entropy is everywhere, guiding every aspect of our lives. Therefore, it is only natural to think that this must somehow be present in financial systems as well. Now we will apply this in our project.

2.2.3 Entropy in Information Theory

Entropy which has been derived from the field of thermodynamics has been adapted into other areas such as information theory. What is Information theory? It is a subfield of mathematics used for quantifying, storing and transmitting information in an efficient manner. The cornerstone of information theory is calculating the amount of information a message carries. It was developed by Claude Shannon in 1948 and is indispensable in the field of communication, cryptography and data compression. Its most important function is to reduce uncertainty. It is highly useful in machine learning and with features like information entropy is used in feature selection, building decision trees, fitting classification models and so on. It also ensures accurate transmission of data in communication networks despite noise.

Entropy comes in Information theory as a measure of uncertainty or disorder in a particular system. This is similar to its physics based counterpart as it should be. In Physics, it is that quantity which describes how energy is dispersed in a system to what extent, while in information theory it represents the unpredictability and disorder of information content of a system. This concept of Information Entropy was introduced by Claude Shannon in 1948 in his paper "A Mathematical Theory of Communication", and is therefore also referred to as **Shannon Entropy**. [10]

Let X be a random variable and $P_X(x)$ be its probability density function (pdf). Its Entropy $H(X)$ can be mathematically defined as

$$H(X) = - \sum_{x \in X} P_X(x) \log P_X(x) \quad (2.2)$$

The derivation to which is given in the following section and also can be found in [10].

2.2.4 Entropy in Finance

An important application of Shannon Entropy is in finance. In financial markets, we can apply Shannon Entropy to measure the unpredictability of returns of a particular asset. It gives us a different way to understand the degree of disorder in the distribution of returns.

We will now try to arrive at Shannon entropy using the classical entropy formula. We know entropy of a system with Ω equally likely microstates is given by:

$$S = k_B \ln \Omega,$$

where Ω is the number of microstates corresponding to a macrostate. For a system of N independent trials (or particles) distributed among n states with probabilities $\{P_1, P_2, \dots, P_n\}$, the most probable configuration has

$$N_i \approx NP_i \quad \text{for } i = 1, \dots, n. \quad (2.3)$$

The number of microstates consistent with these counts is given by the multinomial coefficient

$$\Omega = \frac{N!}{N_1! N_2! \dots N_n!}. \quad (2.4)$$

For large N and N_i , we use Stirling's approximation

$$\ln m! \approx m \ln m - m. \quad (2.5)$$

Thus,

$$\ln \Omega \approx \ln N! - \sum_{i=1}^n \ln(N_i!) \approx (N \ln N - N) - \sum_{i=1}^n (N_i \ln N_i - N_i).$$

Since $\sum_{i=1}^n N_i = N$, the constant terms cancel and we have

$$\ln \Omega \approx N \ln N - \sum_{i=1}^n N_i \ln N_i. \quad (2.6)$$

Replacing each N_i by NP_i ,

$$\ln \Omega \approx N \ln N - \sum_{i=1}^n (NP_i) \ln(NP_i).$$

Now, expanding the logarithm,

$$\ln(NP_i) = \ln N + \ln P_i,$$

so that

$$\ln \Omega \approx N \ln N - \sum_{i=1}^n N P_i (\ln N + \ln P_i).$$

Distributing the sum gives

$$\ln \Omega \approx N \ln N - N \ln N \sum_{i=1}^n P_i - N \sum_{i=1}^n P_i \ln P_i.$$

Since $\sum_{i=1}^n P_i = 1$, the first two terms cancel, leaving

$$\ln \Omega \approx -N \sum_{i=1}^n P_i \ln P_i.$$

Plugging $\ln \Omega$ into the Boltzmann formula,

$$S = k_B \ln \Omega \approx -k_B N \sum_{i=1}^n P_i \ln P_i.$$

Thus, the entropy per trial (or per particle) is

$$\frac{S}{N} \approx -k_B \sum_{i=1}^n P_i \ln P_i.$$

Setting the Boltzmann constant $k_B = 1$ (or absorbing it into the unit definition), we obtain the Shannon entropy:

$$H = - \sum_{i=1}^n P_i \ln P_i \tag{2.7}$$

$$\text{or, } H(X) = - \sum_{x \in X} P_X(x) \log P_X(x) \tag{2.8}$$

High value of entropy suggests uncertainty of the market implying that an asset's return is spread over a large area of outcomes. Conversely, lower value of entropy suggests that the return is more concentrated around some particular values, hence, showing lower uncertainty or disorder.[9]

We consider an example to Understand how the above expression works in real life. Suppose we have a stock that gives us daily returns over a period of one month. We take these daily returns in the form of a set. We will try to quantify the uncertainty in the returns. To do this we firstly divide this range of returns into a fixed number of bins(say 10). For each bin, we calculate the probability P_X of return falling in that region. This creates an estimated probability distribution of the returns.Using Shannon Entropy

formula we have

$$H(X) = - \sum_{x=1} P_X(x) \log P_X(x) \quad (2.9)$$

which helps us to calculate the entropy. If in one case the returns are spread uniformly throughout the bins, P_X would be equal, i.e. 0.1 for 10 bins. In this case, the expression gives us a high entropy showing us a high degree of unpredictability and uncertainty. However, if for the other case we have returns that are concentrated in one or two bins, the probability P_X for those bins would be much higher. Here the expression gives us a low value of entropy, meaning the returns would be predictable.

2.2.5 Entropy in our program

In our program we have used entropy in the following way:

```
def calc_entropy(series):
    #Shannon entropy of the distribution of returns.
    hist, _ = np.histogram(series, bins=10, density=True)
    return entropy(hist) # from scipy.stats
```

We have defined a function named **calc_entropy** to calculate the entropy. The function first creates a histogram from a series of returns using **np.histogram(series, bins=10, density=True)**. The **bins=10** argument divides our data (the series of returns) into 10 bins and computes a normalized histogram. The **density=True** parameter ensures that the sum (or rather the integral over the bins) equals 1, so the resulting values in **hist** represent an approximation of the probability density function (pdf) of the returns.

```
    return entropy(hist)
```

This line calls the **shannon entropy** function from **scipy.stats**. Here, the values of **hist** play the role of the probability P_X (or, more precisely, the

density in each bin). Thus, the entropy returned by the function is a measure of the uncertainty or randomness of the return distribution.

By applying this entropy calculation on a rolling window (in our case, 30 days), the program extracts a time series of entropy values for each stock. This feature is then incorporated into the feature set used by the Random Forest regressor.

2.3 Brownian Motion

2.3.1 Introduction

When beams of sunlight enter our rooms, we see minute dust particles moving in such a manner that it seems as though they are dancing in the air. This dance indicates that there are underlying movements of matter which are not visible to the naked eye. This movements originates from the atoms which themselves moves spontaneously. So wrote the Roman philosopher-poet Lucretius in his scientific poem "On the Nature of Things" (60 BC) on the motion of dust particles, in verses 113–140 from Book II.[11]

2.3.2 Brownian Motion in Physics

Brownian motion happens when tiny particles in a liquid or gas move in a random way because they are constantly hit by surrounding molecules. This effect was first noticed by scientist Robert Brown in 1827 while using a microscope to observe pollen grains in water.[12] He saw that these grains moved in a shaky and unpredictable way. At first, people thought this movement was caused by something biological, but later, experiments showed that non-living particles moved the same way. This proved the motion was not related to life, though its cause was still unknown at that time.

Few papers were published to describe the mathematics behind Brownian Motion by researchers such as Thorvald N. Thiele and later independently by Louis Bachelier. Finally, it was Albert Einstein (in 1905) and Marian Smoluchowski (1906) who gave a solution to the problem.[14]

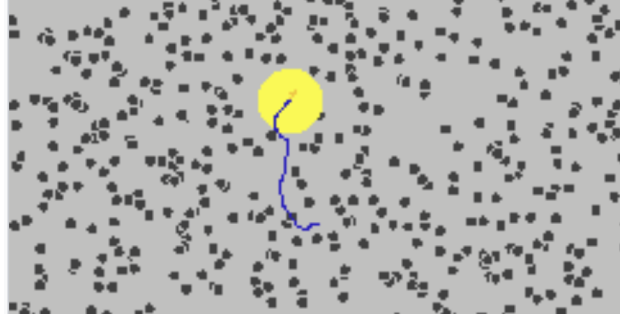


Figure 2.1: Brownian motion[13]

When we dive further into the microscopic level, the movements in fluids molecules are perpetual (never-ending), because of the motion due to thermal energies. When these molecules in motion collide with larger particles, such as pollen grains or dust particles, they impart forces upon these particles. This force makes the particles move, and because the tiny molecules keep colliding with them from different sides, the direction of movement keeps changing randomly. At a certain time, the collisions and the resulting forces are more in one particular direction than the other, while at another instant, the directions are different. This result in the random motion of that particles with time, leading to what we call the Brownian Motion.

Mathematically, Brownian motion can be modelled as a stochastic process. A stochastic process is one in which the outcome at each point in time is a random variable in a system that is evolving with time.[15] One of the standard ways to mathematically model Brownian motion is through the Wiener process. The Wiener process is a mathematical model of a continuous-time stochastic process, meaning it models systems that evolve randomly with time. It is characterized by independent increments and a Gaussian distribution. To put it simply, if we track the position of a particle over time then the changes in position would be random and will follow a normal distribution with a mean of zero. The variance of this distribution increases linearly with time, which gives us the idea that the longer the particle moves, the further it wanders away from its original/initial position.

Let $X(t)$ be the position of a particle at time t . $X(t)$ is a function of time meaning that its position depends on time. The Wiener's Process is characterized by the properties:-[14]

1. The position at $t = 0$ is considered to be zero,

$$X(0) = 0 \quad (2.10)$$

2. The increments $X(t + \Delta t) - X(t)$ are independent and normally distributed.
3. The mean of the increments is zero:

$$\langle X(t + \Delta t) - X(t) \rangle = 0. \quad (2.11)$$

4. The variance of the increments is proportional to the time interval:

$$\text{Var} [X(t + \Delta t) - X(t)] = \Delta t. \quad (2.12)$$

Thus, over a time period t , the displacement $X(t)$ is normally distributed with mean 0 and variance t :

$$X(t) \sim N(0, t). \quad (2.13)$$

where, $N(0, t)$ represents normal distribution with the *Mean* = 0 and the *Variance* = t . This implies that the probability density function of finding the particle at position x at time t is given by:

$$P(x, t) = \frac{1}{\sqrt{2\pi t}} \exp \left(-\frac{x^2}{2t} \right). \quad (2.14)$$

2.3.3 Brownian Motion in Finance

The mathematical theory of Brownian motion has been applied in areas far beyond those of molecular motion. Brownian motion plays a very important role in financial modelling and quantitative finance. Brownian motion is key in understanding financial markets. Stock market, foreign exchange market and so on are all assumed to follow Brownian motion where all the returns and assets are continually changing in the smallest intervals of time with unpredictable amount of change each time. In the view-point of finance, stock prices are often assumed to follow a "random walk," a scenario where the future price is modelled as the current price plus a random change. This idea is supported by the concept of Brownian motion and has led to widely used models, such as the Black-Scholes option pricing model.

In finance, Wiener process, which represents the continuous-time version of the Brownian motion, is a key concept. The changes in position follow a normal distribution, meaning they are random but follow a predictable pattern. On average, there is no net movement in any direction, and the spread of these changes depends on the time step. Let $S(t)$ represent a stock price. According to the Black-Scholes equation, $S(t)$ is modelled as,[16]

$$dS(t) = \mu S(t) dt + \sigma S(t) dW(t), \quad (2.15)$$

where μ is the drift term representing the expected return, σ is the volatility (or standard deviation) of the returns, and $W(t)$ denotes the increment of the Wiener process. This equation captures both the predictable trend and the unpredictable, random fluctuations in stock prices.

The randomness in the Brownian motion provides a natural, physical and mathematical way to represent the uncertainty and risk associated with financial markets. Brownian motion is widely used since randomness inherent in Brownian motion provides a natural way to represent the uncertainty and risk associated with financial markets. to estimate risks, choose investment strategies and investment according to future price paths. It is Brownian motion that has given rise to Monte Carlo simulation techniques in finance.

2.3.4 Brownian motion in our program

We use Brownian motion in our program as a way to capture the randomness and variability of daily price changes. As we have already dealt with earlier, in an ideal Brownian motion, the changes (or increments) over very short time intervals are random, normally distributed with a mean of zero, and their variability (or spread) is proportional to the time period. In our program, we try to model this behaviour by looking at the differences between consecutive daily returns.

```
def calc_brownian(series):
    #Standard Deviation of daily differences as
    #a Brownian proxy
    return np.std(np.diff(series))
```

The function is defined and named as **calc_brown** which has **series** as the accepted parameter. The argument **np.diff** computes the difference between

consecutive elements in the series, effectively giving daily changes in stock returns. The argument `np.std` calculates the standard deviation of these daily changes.

Here we are taking the standard deviation because the standard deviation is a natural measure of the spread (or variability) of a normal distribution and the increments of a Brownian motion are normally distributed. By computing the standard deviation of the daily differences, we get an estimate of how much the returns vary from one day to the next. This is the most simple and computationally efficient method of measurement of the brownian motion

2.4 Volatility

2.4.1 Introduction

Nothing in this world is permanent. Everything is in a state of change. From the tiniest of particles to the largest of objects, all are always going through. Be it the scenery of a place to ourselves (our physical and emotional characteristics), we are always undergoing change. Any particular state/condition of anything is fleeting. They are volatile.

2.4.2 Volatility in Physics and Mathematics

In physics, volatility is defined as the measure of fluctuations or variations in a physical quantity over time. In a physical system, volatility manifests itself as fluctuations in parameters such as temperature, pressure, and energy, which result from the inherent randomness of microscopic interactions.

Particles are always in a state of random motion due to thermal excitations such as molecules in a gas. These motions are responsible for the fluctuations in macroscopic properties, such as pressure and temperature. For gases, the pressure on the walls of a container depends on the number of collisions of the molecules of a gas. These collisions occur randomly and result in measurable fluctuations around an average value. The degree of these fluctuations can be quantified by calculating the variance or standard deviation. This provides a measure of volatility.

An example where fluctuations play a key role is in phase transitions. Near the critical point, where a substance transitions from one phase to another, fluctuations in density or magnetization become pronounced. These fluctuations are not any random noise, but are instantaneous states that reveal important information about the system's behaviour. Such examples are present in different aspects of physics such as distribution of molecular speeds described by the Maxwell-Boltzmann distribution, in condensed matter physics, phenomena such as electrical noise in conductors also known as flicker noise and so on.[17]

2.4.3 Volatility in finance

Volatility is an important idea in finance. It measures how much an asset's price changes over time, often calculated using the standard deviation of its returns. It helps indicate how uncertain or risky an asset is. This is crucial for financial models, especially in managing risks and pricing derivatives.



Figure 2.2: Volatility of Financial Market[18]

Mathematically, volatility is often defined as the standard deviation of returns. Returns can be calculated using either simple returns or log returns. For a given asset with price P_t at time t and price P_{t-1} at time $t - 1$, the simple return R_t is defined as:[19]

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \quad (2.16)$$

Alternatively, the log return, which is widely used due to its additive properties over time, is given by:

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right) \quad (2.17)$$

Once returns are computed, the volatility σ is calculated as the standard deviation of these returns over a certain period. If we have N returns $r_1, r_2, \dots, r_N$, the volatility is:

$$\sigma = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (r_i - \bar{r})^2} \quad (2.18)$$

where $\bar{r}$ is the mean of the returns:

$$\bar{r} = \frac{1}{N} \sum_{i=1}^N r_i \quad (2.19)$$

This tells us how much the returns deviate from their average value. The higher value of σ indicates that the returns are spread widely, meaning the asset price is highly volatile and more unpredictable. Lower σ shows stability.

In portfolio optimization, volatility, is used to calculate the risk of individual assets as well as entire portfolios. The classic mean-variance optimization framework, introduced by Harry Markowitz tries to maximize returns for a given level of risk (or equivalently, minimize risk for a given level of return). In this framework, the portfolio variance is calculated as:[20]

$$\sigma_p^2 = \sum_{i=1}^n \sum_{j=1}^n w_i w_j \sigma_i \sigma_j \rho_{ij} \quad (2.20)$$

where:

- w_i and w_j are the weights of assets i and j in the portfolio,
- σ_i and σ_j are the volatilities of assets i and j ,
- ρ_{ij} is the correlation between the returns of assets i and j ,
- n is the number of assets in the portfolio.[20]

From this expression, we understand that portfolio risk is not only dependent on the individual asset volatilities but also highly dependent on their correlations. By combining assets that are not perfectly correlated, investors can reduce the overall volatility in their portfolio through diversification. In practice, volatility is not constant. It varies over time as a result of changes in market conditions.

2.4.4 Volatility in our program

We have implemented volatility in our program in the following way:

```
def calc_volatility(series):  
    """Standard Deviation as a measure of volatility."""  
    return series.std()
```

As we have seen above, in our program we have measured the volatility of stock returns using standard deviation. Thus we make use of equation (2.18) above. The $N - 1$ of the equation is replaced by N as we are considering the entire population. Here, r_i is the daily log returns, $\bar{r}$ is the mean of the series and N is the number of data points in the rolling window. If the standard deviation is high, it means the price changes a lot. If it's low, the price remains more stable.

2.5 Simple Harmonic Motion (SHM)

2.5.1 Introduction

Everything around us is in a state of constant motion. From the largest of things like planets, stars and galaxies to the tiniest of particles like subatomic particles, everything is in constant motion. All these forms of motion of different bodies have different properties. Some move slowly, some very fast. Some never return to their initial positions while others vibrate about their mean position. Simple Harmonic motions is one among such varied types of motion which is guided by the simplest properties.

2.5.2 Simple Harmonic Motion in Physics

Simple Harmonic Motion (SHM) is a type of repeated motion where an object moves back and forth around a central position. This happens because a force pulls it back toward the centre, and the farther it moves away, the stronger the force gets. The direction of this restoring force is always towards the equilibrium position. The movement follows a smooth, wave-like pattern and continues indefinitely. SHM is defined by a direct relationship between the restoring force and displacement.

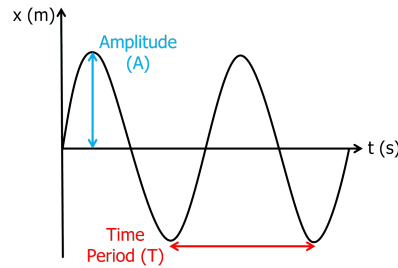


Figure 2.3: SHM graph[21]

Let us consider, a mass attached to one end of a spring, of spring constant $k \text{ N/m}^2$. The other end of the spring is connected to a rigid support. If the system is at rest in its balanced position, no force acts on it. However, if the mass is moved away from this position, the spring applies a force to bring it back. This force follows Hooke's Law, which explains how a spring's force depends on its stretch or compression. The force F is given by

$$F = -kx \quad (2.21)$$

where x is the displacement from mean position.[22] The negative sign in the equation shows that the force always acts in the opposite direction of the displacement from the equilibrium position, pulling the object back to its original state.

When we let go of a mass/object that has been displaced from its mean position, it experiences an acceleration in the direction of the equilibrium position. The potential energy that has been stored in the body gradually turns into kinetic energy. Even after the mass/body reaches the equilibrium

position (max kinetic energy), its momentum carries it past, resulting in the formation of restorative forces in the direction opposite to that of motion (towards the mean position). The body again comes to a halt, regains max potential energy and is accelerated in the opposite direction, reversing its motion. This leads to a cyclic process that results in oscillations that continue indefinitely in an ideal, frictionless environment.

The motion of a system can be described by a sinusoidal function. For a mass-spring system, the displacement as a function of time is given by,[22]

$$x(t) = A \cos(\omega t + \phi) \quad (2.22)$$

where A is the amplitude (maximum displacement), ω is the angular frequency (related to the period of the oscillation) and ϕ is the phase constant, which depends on the initial conditions of the motion.

The angular frequency ω is related to the mass m and the spring constant $k(x)$ as

$$\omega = \sqrt{\frac{k}{m}} \quad (2.23)$$

this equation shows that the frequency of oscillations depend on the physical features of the system, such as stiffness of the spring and the mass of the object. The choice of sinusoidal function as sine or cosine is merely a matter of convention. A general rule is, when the displacement is maximum at $t = 0$ we use cosine and when the displacement is 0 at $t = 0$ we use sine, to make further calculations simpler.

No matter the interchange of energy in the system, the total mechanical energy of the system remains constant given there is no damping force or friction.

$$E = \frac{1}{2}kA^2. \quad (2.24)$$

2.5.3 Simple Harmonic Motion in Finance

Simple Harmonic Motion, though originally meant for mechanical systems, finds a lot of usage in various fields. One such application is in the field of finance, particularly in modelling stock market. The principles of SHM have

found great application in the field of finance. The cyclical patterns, mean-reverting tendencies, and oscillatory behaviour which are often observed in asset prices and market indices can be understood, predicted and modelled based on principles of SHM.

The financial market is filled with random fluctuations over a short period of time. However, if we look at it carefully over a longer range of time, financial markets are not always random. The prices do not move randomly in an unpredictable manner. Rather, they often exhibit cyclical patterns influenced by investor sentiment, economic cycles, and market corrections. Phenomena such as **mean reversion** are observed. It is a theory in finance which suggests that asset prices, after an extreme move (extreme losses or gains), tend to revert back to their long-term average/equilibrium level or value. This cyclical behaviours can be though to be similar to the oscillations in Simple Harmonic Motion, where these deviation from the average value lead to "restoring forces" which drive the price back to the mean. In this regard, SHM provides a mathematical framework for understanding oscillatory market behaviour and helps identify periods of overvaluation or undervaluation.

We use SHM-inspired models to analyse mean-reverting stocks, where prices tend to oscillate around an equilibrium value. By analysing historical price movements, we approximate stock oscillations and determine potential reversals.

Mathematically we know SHM are represented by equations of the form,

$$x(t) = A \cos(\omega t + \phi) \quad (2.25)$$

where A is the amplitude (maximum displacement), ω is the angular frequency (related to the period of the oscillation) and ϕ is the phase constant, which depends on the initial conditions of the motion. In finance, these parameters A , ω and ϕ can be loosely related to the amplitude of price fluctuations, the speed of oscillation, and the timing of market cycles. Even if the market does not follow this sinusoidal motion, the underlying forces that drive prices back to the mean from the extremes are useful. With further investigation of historical data about the market, the cyclical patterns that resemble the oscillatory motion described by Simple Harmonic Motion can be identified. Also some analysts employ methods of moving averages and

oscillators (like the Relative Strength Index) to determine whether a stock price is deviating significantly from its average value. If such a case is discovered then, according to the properties of SHM, it suggests that a corrective move might soon occur.[19]

2.5.4 Simple Harmonic Motion in our program

We have used SHM in our program in the following way:

```
def calc_shm(series):  
    #Range of values (max - min) as a simple measure  
    #of oscillation.  
    return np.max(series) - np.min(series)
```

We define a function named **calc_shm** which captures the amplitude of price fluctuations over a given window. The function computes the range of the series taking the difference between its maximum and minimum values as it passes the value given by **return np.max(series)-np.min(series)**. This measure quantifies the extent of oscillation or variability within that period. SHM is characterised by their extent or amplitude (maximum, deviation from its mean position). This difference serves as the proxy for for this amplitude, reflecting how much the returns (or prices) vary within the rolling window. Thus, when SHM is used alongside other concepts like entropy, Brownian motion and volatility, it provides additional insight into the dynamics of price changes. It helps the machine learning model understand the extent of fluctuations, which might be related to risk or market momentum. Thus, it helps in feature enrichment.

Chapter 3

The Program

What is a Computer Program? A computer program is a collection of commands written in a programming language to perform a specific task or to get a particular result. It is the means by which we try to use computing resources in order to gain results that are otherwise difficult, if not impossible, to obtain by human endeavour. For our project, we have prepared a program that can integrate Physics in Machine Learning and help in Risk Management in Stock Markets and Portfolio Optimization. For our programming we will be using **Python**.

3.1 Python Libraries Imported

Python is one of the most commonly used programming language because of its simplicity, readability, and versatility. Python is a high-level, interpreted language which has easy-to-understand syntax which makes it highly user friendly. However, this is not all. Another important reason that python is so widely used is because of its extensive collection of libraries available for various applications, from data analysis and machine learning to scientific computing and web development. A Python library is essentially a collection of pre-written code, modules or files that contains functions, classes, and constants to perform specific tasks or operations, helping developers and programmers save time and effort by avoiding the need to write everything from scratch.

Python Libraries helps in making the programs simpler by providing pre-written and verified code that also helps improve efficiency. Furthermore,

the large community of programmers help in improving and upgrading the codes. The codes can be used by typing the "import" command.

Without these libraries our programming would have been next to impossible. We have imported several python libraries for the purpose of data input, data analysis, machine learning, pre-processing, algorithm implementation and model evaluation. The libraries we imported are as follows:-[23]

Import Statement	Description
<code>import numpy as np</code>	NumPy provides powerful numerical operations and support for multi-dimensional arrays, enabling efficient mathematical computations.
<code>import pandas as pd</code>	Pandas offers robust data manipulation and analysis tools using DataFrames, ideal for handling structured data like time series.
<code>import matplotlib.pyplot as plt</code>	A graphing library that helps create both static and interactive charts, including line graphs, bar charts, and scatter plots.
<code>from sklearn.model_selection import train_test_split</code>	train_test_split: Splits datasets into training and testing subsets.
<code>from sklearn.preprocessing import StandardScaler</code>	StandardScaler: Adjusts features by subtracting the mean and scaling them to have a variance of one.
<code>from sklearn.ensemble import RandomForestRegressor</code>	RandomForestRegressor: An ensemble method for regression tasks that combines multiple decision trees.
<code>from scipy.stats import entropy</code>	entropy: Calculates Shannon entropy, which quantifies uncertainty in a distribution.
<code>import requests</code>	Allows making HTTP requests to external APIs, useful for fetching data from services like Tiingo or OpenWeather.
<code>import os</code>	Lets us interact with the operating system, including file and directory management.

Import Statement	Description
<code>import time</code>	Provides time-related functions, such as adding delays to manage API call frequency.

One of the most important component of our program is the machine learning modules and its usage for prediction in our program. We have used the modules: **train_test_split**, **StandardScaler** and most importantly the **RandomForestRegressor**. We have explained them below.

3.2 train_test_split

Train-test split is an important process in machine learning that helps to access how well a machine learning model will work with different and new data. How it works is very simple. It divides the data-set into two parts: one portion which is used for training the machine learning model(training set) and the other which is used to test the trained model and evaluate the model's performance(test set). This partition is very crucial because even if the model works exceptionally well on the train data-set, it might just be because it is memorising the data. This phenomenon is called overfitting. It dose not actually learn the underlying patterns. Testing the model on a separate data set helps us get a more realistic estimate of its predictive power.

In Python's scikik learn module, `train_test_split` function is used for exactly this purpose. It helps to randomly partition the data-set into training and testing subsets. It ensures both of the subsets are representative of the overall data. This randomness is important to avoid any bias in the split. Its syntax is given below,[4, 24]

```
train_test_split(*arrays, test_size, train_size,
random_state)
```

An example is given below

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Thus we see that `"*arrays"` is the data-set that we want to split. `"test_size"` defines by what percentage the data-set is to be divided (20% in above and 80% for training). `"random_state"` parameter ensures consistent shuffling before splitting the data.

Using a train-test split is very useful. It helps to measure a model's predictive ability without any bias as it is working on unseen data to test. By training the model on one set of data and testing it on another, we can detect overfitting and make adjustment. It is one of the first steps in verifying a predictive model as it helps simplify the process of dividing our data into training and testing sets.

3.3 StandardScaler

StandardScaler is an essential tool in machine learning. It works by making the data uniform through mean removal and scaling to unit variance. In many data-sets some variables may range from 0 to 1 while others range from a thousand to a crore. Such differences in data-set ranges can bias algorithms, particularly those that use distance metrics (like k-nearest Neighbors or support vector machines) or gradient-based optimizers. Therefore, in such cases, we make use of StandardScaler which transforms the data so that each feature has a mean of 0 and a standard deviation of 1.

StandardScaler is based on the formula,[4, 24]

$$z = \frac{x - \mu}{\sigma} \quad (3.1)$$

Here, x represents an original feature value, μ is the mean of the feature values in the training set, and σ is the standard deviation. The result, z is the standardized value.

StandardScaler works in a very simple way. First, the scaler is fitted to the data that is in train. After this it calculates the mean and standard deviation for each feature. Then, based on these computed values, it transforms the training data. Finally, the same transformation is applied to the data in test using the statistics from the data in train to maintain consistency.

Data-set standardization is very important in machine learning. It helps improve the overall model performance which converges faster. It makes sure

that all the features or the input data contributions are equal. Features with higher magnitudes are prevented from overpowering the learning process. Standardizing features can also help prevent numerical issues by making sure that features with vastly different scales do not get processed together. Both the training and testing data-set are standardized to the same extent which helps in better model generalization. This consistency is vital because the model should make predictions on test data that has the same distribution as the training data. By standardizing features, it creates a uniform scale that allows learning algorithms to perform more effectively and efficiently. This ensures that the model is not biased towards features with larger numerical ranges, ultimately leading to more robust and reliable predictions in real-world applications.

3.4 RandomForestRegressor

RandomForestRegressor is a machine learning model which belongs to ensemble learning family. In physics, ensemble is a statistical mechanics concept which refers to a state of a system that is the same macroscopically but is different microscopically. Thus, it is a machine learning module which learns from different arrangements of similar looking data-sets. It is primarily used for regression tasks, which means it predicts a continuous numerical value based on input features. This model of machine learning is built to work upon decision trees. It enhances these decision trees by constructing multiple such decision trees and combining them to gain output. This method reduces overfitting, increases accuracy, and provides more stable predictions compared to using a single decision tree.

The RandomForestRegressor follows a technique called bagging. In this method, multiple models are trained on different parts of the data to enhance predictive accuracy. Each decision tree in the "forest" makes its own prediction, and the final outcome is determined by averaging the predictions of all decision trees. This approach helps in understanding complex patterns in data while ensuring good generalization.

The Random Forest algorithm is made up of multiple decision trees, where each tree is trained on a randomly chosen portion of the dataset. These trees are built independently by the algorithm and each one of them is trained using different feature combinations and approach to ensure diversity. The

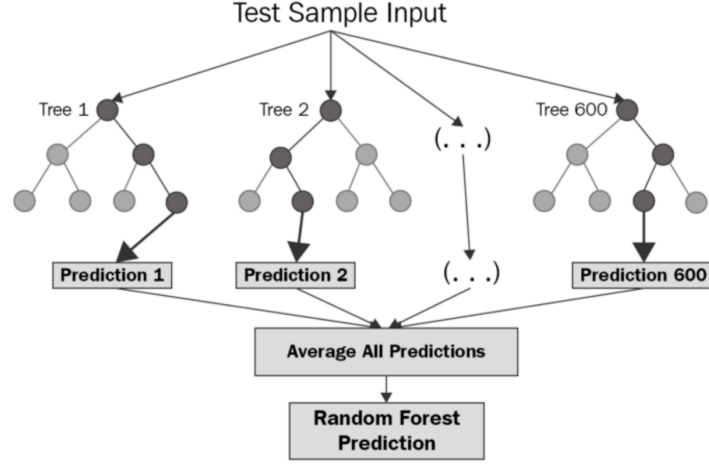


Figure 3.1: Representation of RandomForestRegressor[25]

overall prediction is obtained by averaging the individual decision tree predictions, which helps smooth out variations and errors.

Mathematically, the final predicted value $\hat{y}$ is given by[26, 4]

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x) \quad (3.2)$$

where,

- T is the total number of decision trees.
- $h_t(x)$ is the prediction from the t^{th} tree.

By averaging the predictions of multiple trees, RandomForestRegressor minimizes the variance and provides a better estimate than a single decision tree.

We use RandomForestRegressor in our program to predict stock returns based on various financial factors and physics-inspired features. The dataset we provide consists of historical stock price data and returns. The model learns to predict stock returns based on various financial and physics-inspired features. The model learns patterns in this data to forecast future returns.

Why do we use RandomForestRegressor over other predictive models

We use RandomForestRegressor over other models because it provides high accuracy, robustness to overfitting, and handles non-linearity well. It is a learning technique that constructs multiple decision trees and combines their predictions by averaging, which helps minimize variance and enhance generalization. Unlike linear models, it captures complex relationships without requiring feature engineering. It also works well with missing data and large datasets, making it a reliable choice for many regression tasks.

3.5 Pandas

Pandas is a widely used open-source Python library designed for data manipulation and analysis. It offers structures for data such as Series and DataFrame, which help in efficiently managing this data which is structured. A Series represents a one-dimensional labelled array, whereas a DataFrame is a two-dimensional table. The name "Pandas" comes from the term "Panel Data," which refers to a dataset.

Pandas simplifies data cleaning, transformation, and exploration in Python. It allows users to read and write data from various formats like CSV(Comma Separated values), Excel, SQL databases, and JSON. Common operations that can be easily done with the help of Pandas include filtering, sorting, merging, reshaping, and handling missing values. Another feature of Pandas that makes it very powerful is its integration with NumPy and Matplotlib, making it ideal for data analysis and visualization. Also it supports vectorized operations, allowing for efficient computation without explicit loops.

Pandas is very often used in finance, machine learning, and scientific computing, enabling quick data preprocessing and feature extraction. It supports time-series analysis, making it particularly useful for stock market predictions and trend analysis.

We have used Pandas in our program by importing it as:[27]

```
import pandas as pd
```

The various functions of Pandas that we have used are as follows:

Table 3.2: List of Pandas Functions Used in Our Programs and Their Descriptions

Pandas Function	Description
<code>pd.DataFrame()</code>	Creates a DataFrame object from various data structures (e.g., dictionaries, JSON), converting raw data into a structured table format.
<code>pd.to_datetime()</code>	Converts date strings into datetime objects, ensuring that date information is properly formatted for time-series operations.
<code>DataFrame.set_index()</code>	Sets a specified column (often the date column) as the DataFrame index, which is crucial for aligning and processing time-series data.
<code>DataFrame.sort_index()</code>	Sorts the DataFrame by its index, ensuring the data is in chronological order.
<code>DataFrame.to_csv()</code>	Writes the DataFrame to a CSV or text file; used here to cache fetched data locally in a tab-delimited format.
<code>pd.concat()</code>	Combines multiple DataFrames along a specified axis (e.g., columns) to merge individual stock data into a single DataFrame.
<code>DataFrame.dropna()</code>	Removes rows or columns with missing values, ensuring that subsequent analyses and calculations are performed on complete data.
<code>DataFrame.pct_change()</code>	Computes the percentage change between consecutive elements, used to calculate daily returns from stock prices.

(Continued on next page)

(Continued from previous page)

Pandas Function	Description
<code>DataFrame.diff()</code>	Calculates the difference between consecutive rows, providing an approximation of Brownian motion increments in our returns.
<code>DataFrame.head()</code>	Returns the first few rows of a DataFrame, which is useful for previewing the data and verifying that it has been loaded and processed correctly.

3.6 cache_data.py

As the name suggests it is a python file that contains codes of our program for cache data. We have a separate program for downloading and storing data relevant for our program. The entire program code is as follow

3.6.1 Importing Modules

We begin by importing the relevant modules

```
import os
import time
import requests
import pandas as pd
```

as discussed earlier these modules help us in various ways. **os** allows the program to interact with the operating system. **time** allows us to use time-related functions such as adding delays. **requests** allows making HTTP requests to external APIs to fetch data. **pandas** allow us to handle structured data well.

3.6.2 Passing the API and the base URL

```
API = "X" # Replace X with Tiingo API key
BASE_URL="https://api.tiingo.com/tiingo/daily/{ticker}/prices"
```

Here we have assigned our API key and the website we use to retrieve the historical stock data to string variables API and BASE_URL.[7]

3.6.3 Defining Function `fetch_stock_data`

Now we have defined the function to fetch data from the link provided. The function is defined as

```
def fetch_stock_data(ticker, start_date="2023-01-01",
                    end_date="2025-03-10"):
    url = BASE_URL.format(ticker=ticker)
    url = f"{url}?startDate={start_date}&endDate={end_date}"
    url = f"{url}&token={API_KEY}"
    response = requests.get(url)

    if response.status_code == 200:
        data = response.json()
        .
        .
        .
    else:
        print(f"Error fetching {ticker}: {response.json()}")
        return None
```

The `fetch_stock_data` function is coded to retrieve historical stock data for a given ticker/symbol from the Tiingo API and process it into a pandas DataFrame. Its parameters is explained as follows:

- **ticker** is the stock symbol(eg GOOG)
- **start_date** the beginning of the date range for which data is requested.
- **end_date** the end of the date range

The next two lines of code, i.e.

```
url = BASE_URL.format(ticker=ticker)
url = f"{url}?startDate={start_date}&endDate={end_date}
        &token={API}"
```

is used for the construction of the URL. The first line adds the respective symbol/ticker (GOOG, APPL, AMZN, and so on) to the URL address. The second line adds the start date, end date and the API key. The resulting URL will look like **https://api.tiingo.com/tiingo/daily/AAPL/prices?startDate=2023-01-01&endDate=2023-12-31&token=your_api_key**.^[7]

```
response = requests.get(url)
```

This is a method to send an HTTP GET request to the constructed URL. The API then responds with a JSON data-set containing the historical data if the request is successful.

The next line:

```
if response.status_code == 200:
```

checks if the API call was successful. If it is not successful, a message "Error fetching ticker" (ticker being a particular company) would display and none would be returned as the else block would be activated. If it was successful then the JSON(JavaScript Object Notation) is converted into python object.

```
data = response.json()
```

Now, the function checks whether data is returned

```
if data:
```

If data exists, it continues; otherwise, it prints a message and returns none.

```
df = pd.DataFrame(data)
```

The JSON data is converted to a pandas DataFrame. This allows us to easily understand and manipulate data using pandas. The date column is first converted to datetime objects then it is set as DataFrame index and sorted to ensure chronological order:

```
df["date"] = pd.to_datetime(df["date"])
df.set_index("date", inplace=True)
df.sort_index(inplace=True)
```

Now we have:

```
df = df[["adjClose"]]
df.rename(columns={"adjClose": "Adj Close"},inplace=True)
```

Here, the first line of code creates a new dataframe having only the "adjclose" column. It filters the DataFrame to keep only the column named "adjclose". It is useful when we need only one column for further analysis. The second line then renames the column as "Adj Close". The "inplace=true" argument makes sure that the change is applied directly to the existing DataFrame Without having to assign it to a new variable.

The remaining lines of the function definition handles the exceptional and error cases.

3.6.4 Defining function cache_stock_data

We define another function to save the fetched data. The function is

```
def cache_stock_data(ticker, start_date="2023-01-01",
                    end_date="2025-03-10"):

    filename = f"{ticker}_data.txt"
    if os.path.exists(filename):
        print(f"Data for {ticker} is already cached in
              {filename}.")

        return
    else:
        print(f"Fetching data for {ticker} from Tiingo...")
        df = fetch_stock_data(ticker, start_date, end_date)
        if df is not None:
            # Save as tab-delimited text file
            df.to_csv(filename, sep='\t')
            print(f"Cached data for {ticker} to {filename}.")
        else:
```

```
print(f"Failed to fetch data for {ticker}.")
```

The function **cache_stock_data** is designed to check whether historical data from a particular ticker, i.e. company, is already saved(cached) in a text file. If the data has not already been cached, the function uses the **fetch_stock_data** function to fetch the data from the Tiingo API and save it locally. As the function `fetch_stock_data`, above, `cache_stock_data` has the same parameters.

```
filename = f"{ticker}_data.txt"
```

This line dynamically creates the filename based on the ticker.

```
if os.path.exists(filename):
    print(f"Data for {ticker} is already cached in
                                                {filename}.")
    return
```

Here the function checks if the file with such name exists or not with the help of `"os.path.exists(filename)"` command. If such a file already exists, it displays the message "Data for ticker is already cached in filename" and exists the function. If the file does not exist then it proceeds to fetch data by calling a function `fetch_stock_data` and prints a message to indicate that it is fetching the data.

```
else:
    print(f"Fetching data for {ticker} from Tiingo...")
    df = fetch_stock_data(ticker, start_date, end_date)
```

After fetching, the function checks if `df` has data. If it does, then it saves the data file to a text file using the tab-delimited formatting (the values in each row of your file are separated by a tab character instead of comma or another delimiter).

```
if df is not None:
    # Save as tab-delimited text file
    df.to_csv(filename, sep='\t')
    print(f"Cached data for {ticker} to {filename}.")
else:
    print(f"Failed to fetch data for {ticker}.")
```

In case, if the fetching fails then an error message is generated.

This is the two functions that we have in this program. Finally we have:

```
tickers = ["AAPL", "MSFT", "GOOG", "AMZN"]
```

Here, "tickers" is a list having the symbols of the companies whose data we want to import. The contents of the list can be anything.

```
for ticker in tickers:
    cache_stock_data(ticker)
    time.sleep(2)
```

This is a loop which iterates over a list of stock tickers, fetching and caching data for each one while adding a 2-second pause between each call. For each ticker, Cache_stock_data function is called. Then, the "time.sleep(2)" command helps pause each execution by 2 seconds. This delay helps avoid exceeding the API's rate limits, which could otherwise lead to errors or temporary bans if too many requests are made too quickly. This concludes the program cache_data.py .

3.7 dissertation.py

dissertation.py is the main program on which our project is based. It deals with retrieving the data that we have so meticulously stored, using equations and calculations derived from physics on it, running the data through the machine learning model that has been inspired from physics, getting the results and drawing the conclusions. This is the program where we have actually tried to apply our physics inspired machine learning model on real world historical stock data to verify how well it can predict and help optimize portfolios. The program is explained below:-

3.7.1 Importing relevant modules

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

We begin by importing relevant modules. We have already dealt with **os** and **pandas** in the previous program. **numpy** provides us with powerful numerical operators and support for multidimensional arrays. **matplotlib.pyplot** helps us create static and interactive visualizations such as line charts, bar plots, and scatter plots.

```
# scikit-learn imports
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
```

Next we have imports from Python's Scikit-learn module. An explanation of these commands have been provided earlier.

```
# Using scipy's entropy function for the 'entropy' feature
from scipy.stats import entropy
```

For using entropy in our program we have imported the entropy function from scipy. The entropy() function can be used to compute the Shannon entropy of a discrete probability distribution.

3.7.2 Loading the Cached Data

```
tickers=["AAPL", "MSFT", "GOOG", "AMZN"]
dataframes = []
for ticker in tickers:
    filename = f"{ticker}_data.txt"
    if os.path.exists(filename):
        print(f>Loading data for {ticker} from {filename} ...")
        # Read the tab-delimited file into a DataFrame
        df = pd.read_csv(filename, sep='\t', index_col='date',
```

```

                                parse_dates=True)
    # Rename 'Adj Close' column to the ticker symbol to
                                avoid confusion
    df.rename(columns={"Adj Close": ticker}, inplace=True)
    dataframes.append(df)
else:
    print(f"File {filename} not found. Skipping {ticker}.")

```

Firstly, we store the various stock symbols in the list `tickers` and initialize the list `dataframes`.

```

for ticker in tickers:
    filename = f"{ticker}_data.txt"

```

We then have a for loop which iterates for all the stock symbols which represent individual companies. Inside this for loop we begin with assigning a variable which has a dynamic variable name `"ticker_data.txt"` of files which in the previous program we had dynamically created to store the stock data of individual companies. Then we code an if-else block as under:

```

if os.path.exists(filename):
    print(f>Loading data for {ticker} from {filename} ...")

```

It checks if the file `"ticker_data.txt"` exists. If it does it means the stock data for that particular ticker had been saved before. It prints the message `"Loading data for {ticker} from {filename} ..."`

```

df = pd.read_csv(filename, sep='\t', index_col='date',
                  parse_dates=True)

```

Here, the program reads the tab separated (`\t`) text file and saves it in a pandas dataframe. The argument `index_col='date'` sets the date present in the data as the index. The argument `parse_dates=True` ensure that the date column is treated as actual datetime values.

```

df.rename(columns={"Adj Close": ticker}, inplace=True)
dataframes.append(df)

```

In the original data-set there is a column called `"Adj Close"` (The one we had renamed in the previous program) which contains values of the adjusted

closing price. Here, we are renaming this column to the ticker symbol (i.e. AAPL, MSFT, GOOG and so on). This simplified working with multiple stocks in the same data-set. The next line appends the dataframe 'df' for a particular ticker to a list called "dataframes" that we had initialized before.

If the filename does not exist, then the else block is activated which prints a message saying file not found and skips to the next ticker. Also if there is no data in the dataframe due to some error we get another error message due to:

```
if not dataframes:
    raise ValueError("No cached TXT files found. Make
                      sure you ran the caching script.")
```

3.7.3 Combining the Dataframes

We combine the data frame because portfolio optimization requires analysing multiple assets together. Also having a combined data helps us have a complete set of data for all the dates. We can clearly see which data is absent and therefore deal with them. It also helps us have a consistent data-set and helps to visualize the market better.

```
# Combining all DataFrames side-by-side (axis=1), using the
#date index
price_data = pd.concat(dataframes, axis=1)
# Drop any rows that have missing data for any ticker
price_data.dropna(inplace=True)

print("\nCombined Price Data (first 5 rows):")
print(price_data.head())
```

Dealing with the above snippet line by line.

```
price_data = pd.concat(dataframes, axis=1)
```

This line combines or merges all the dataframe stored in the list "dataframe" with the help of the pandas keyword **pd.concat**. The **axis=1** keyword aligns the DataFrame side by side (in the form of columns) in such a way that they have one shared index, which is their date. For example, let us consider two

dataframes, having data for tickers, say AAPL and GOOG. The resulting DataFrame has two columns, one for each ticker, with their rows aligned by their shared date.

```
price_data.dropna(inplace=True)
```

This line removes any rows from the dataframe which has missing or NaN (not a number) values. The **inplace=True** argument makes sure that the change happens to that dataframe, in this case being **price_data**, without having to create a new variable/object. This ensures that every row has complete data for every ticker, which is very important for our calculations ahead.

```
print("\nCombined Price Data (first 5 rows):")
print(price_data.head())
```

As the **print** line suggests, this line prints the first 5 rows of our final combined dataframe providing a quick look at the structure and content of our dataset.

3.7.4 Calculating Daily Log returns

Daily log returns is widely used in finance to measure price change of assets over time. The expression is given below,[20, 16]

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right) \quad (3.3)$$

There are a number of reasons why we calculate the daily log returns. Log returns are additive over time. we can simply add them to get returns over a period of time. Log returns tend to follow normal distribution which is beneficial for machine learning analysis and statistics. It also makes sure that while data processing there is no bias over goods that are usually expensive over goods that are usually cheaper. Log returns also provide a more stable measure of volatility compared to simple returns.

```
returns = np.log(price_data / price_data.shift(1)).dropna()
print("\nDaily Log Returns (first 5 rows):")
print(returns.head())
```

The argument `price_data.shift(t)` shifts the entire data-set down by one row, aligning each price with its previous day's price. Thus, in the first line of the snippet we have equated the dynamic variable returns with the expression of daily log returns that we earlier discussed. `dropna()` is present because it will remove the first value as $t - 1$ would be NaN. The next two lines is used to displays the first five rows of the calculated daily log returns for each ticker.

3.7.5 Defining Physics-inspired features

Now comes one of the most important component of our program where we have defined features of physics which we will use to train our machine learning model for risk management and portfolio optimization.

Physics Function definition

The codes are as follows:

```
def calc_volatility(series):
    """Standard Deviation as a measure of volatility."""
    return series.std()

def calc_entropy(series):
    """Shannon entropy of the distribution of returns."""
    hist, _ = np.histogram(series, bins=10, density=True)
    return entropy(hist) # from scipy.stats

def calc_brownian(series):
    """Standard Deviation of daily differences
                                   as a Brownian proxy."""
    return np.std(np.diff(series))

def calc_shm(series):
    """Range of values (max - min) as a simple
                                   measure of oscillation."""
    return np.max(series) - np.min(series)
```

we have dealt with how these expressions were derived before. Moving on to the next segment.

Inputting a rolling window

```

#Computing these features over a rolling window of 30 days.
window_size = 30

# Create an empty DataFrame to store features
features = pd.DataFrame(index=returns.index)

# For each ticker, compute each feature using a 30-day rolling
                                                                    window
for ticker in tickers:
    # Rolling standard deviation for volatility
    features[f"{ticker}_vol"] = returns[ticker].rolling
                                (window_size).apply(calc_volatility, raw=True)
    # Rolling entropy
    features[f"{ticker}_ent"] = returns[ticker].rolling
                                (window_size).apply(calc_entropy, raw=True)
    # Rolling Brownian measure
    features[f"{ticker}_brow"] = returns[ticker].rolling
                                (window_size).apply(calc_brownian, raw=True)
    # Rolling SHM measure
    features[f"{ticker}_shm"] = returns[ticker].rolling
                                (window_size).apply(calc_shm, raw=True)

```

We have set the **window_size** to 30. this means that the program will use a 30-day window of historical log returns for each feature. In the 2nd line we are crating another data frame named features. With the argument **pd.DataFrame(index=returns.index)** we are setting the its index to match the dates (index) of the returns data frame.

```
for ticker in tickers:
```

Now we define a for loop which will loop for each ticker/stock symbol. Within this for-loop, each line will apply a custom function for each of the physics inspired feature. It will be applied to a 30-day rolling window of the log returns for each ticker. We have:

```

features[f"{ticker}_vol"] = returns[ticker].rolling
                                (window_size).apply(calc_volatility, raw=True)

```

- Calculates a measure of volatility (usually standard deviation) over the past 30 days of returns.
- Stores the result in a column named **{ticker}_vol**

```
features[f"{ticker}_vol"] = returns[ticker].rolling(
    window_size).apply(calc_entropy, raw=True)
```

- Computes entropy, a measure of unpredictability, over the same 30-day window.
- Stores the result in a column named **{ticker}_ent**

```
features[f"{ticker}_vol"] = returns[ticker].rolling(
    window_size).apply(calc_brownian, raw=True)
```

- Applies a custom Brownian motion-related function to assess how the returns behave over the same window.
- Stores the result in a column named **{ticker}_brow**

```
features[f"{ticker}_vol"] = returns[ticker].rolling(
    window_size).apply(calc_shm, raw=True)
```

- Computes a simple harmonic motion (SHM) measure, or some other custom indicator, over the 30-day window.
- Stores the result in a column named **{ticker}_shm**

The **raw=True** argument allows pandas to pass a Numpy array instead of a series to each function. This makes it faster if our functions are designed to handle arrays directly.

The resulting **features** data frame will have the same row index as **returns** data frame, but it contains 4 new columns per ticker/stock symbol.

- **{ticker}_vol**
- **{ticker}_ent**
- **{ticker}_brow**
- **{ticker}_shm**

Thus, as we have 4 tickers we will have **16 new columns**.

3.7.6 Defining Target Variables

This portion of our code **defines the target variable** for our predictive model. For our target, we will **predict the "average 5-day-ahead return" across all tickers**. In other words, for a day D , the target we are taking is the mean of returns, across all tickers, on days **$D+1$ to $D+5$** . We have achieved this by simply **shifting the entire data frame by -5** and taking the mean across the columns. The code is as follows:

```
target_5day_ahead = returns.rolling(window=5).mean().
                        shift(-4).mean(axis=1)
```

For each ticker/stock symbol, we use the argument given by the syntax **`returns.rolling(window=5).mean()`** which computes the rolling average (mean) of the returns over a 5-day window. If you have daily returns, it takes the average of 5 consecutive days. For example, for days $D, D+1, D+2, D+3, D+4$, it calculates:[28]

$$\text{Rolling Mean}_{D+4} = \frac{r_D + r_{D+1} + r_{D+2} + r_{D+3} + r_{D+4}}{5}$$

The argument **`shift(-4)`** moves the calculated rolling averages upward by 4 rows (days). This is done to align the rolling average so that it corresponds to the first day of the 5-day window rather than the last. After shifting, each row still has separate values for each ticker. The argument **`mean(axis=1)`** computes the mean across the row ($\text{axis}=1$), i.e. all tickers for that day.

```
target_5day_ahead.dropna(inplace=True)
```

This line Removes any rows in the target variable that contain NaN values. These NaNs can appear because shifting by -5 days leaves missing data at the end of the DataFrame.

```
features = features.loc[target_5day_ahead.index].dropna()
```

This argument **`features.loc[target_5day_ahead.index]`** makes sure that our features data frame has rows corresponding to only those indices (dates) that are present in the cleaned target. The argument **`dropna()`** makes sure that any dates with missing feature values are also removed.

```
target_5day_ahead = target_5day_ahead.loc[features.index]
```

This line re-aligns the target data frame to exactly match the dates in the features data frame. This ensures that each row in our model has both a set of features and a corresponding target value.

```
print("\nFeatures DataFrame (first 5 rows):")
print(features.head())

print("\nTarget (first 5 values):")
print(target_5day_ahead.head())
```

These lines print the first 5 rows of the **features** data frame and the **target** data frame. From here we can inspect the computed features and get an idea of the average 5-day-returns.

3.7.7 Train-Test Split

```
# We'll split into 80% training data and 20% testing data.
X_train, X_test, y_train, y_test = train_test_split(
    features,
    target_5day_ahead,
    test_size=0.2,
    random_state=42
)
```

We have discussed **train_test_split** at length earlier the parameters of the train-set function is as follows:-

features

- This data frame contains all the independent variables which act as predictors.
- They are the computed derived from physics principles for each ticker

target_5day_ahead

- This data frame represents the dependent variables that we are trying to predict. It is the average 5-day-ahead returns.
- It is created by processing historical return data to reflect future performance.

test_size=0.2

- Allocates 20% of the entire data set as the test set for the predictive model. It is a flexible choice and can be more or less.
- The remaining 80% is used to train the predictive model.

random_state=42

- It is essentially a seed for the random number generator used in shuffling the data before splitting.
- It is crucial for reproducibility. We can recreate the exact values for the experiment. it ensures we get the same split.

The **train_test_split** function provides us with 4 output objects/variables.

Variable	Description
X_train	Contains 80% of our features that will be used as predictors to train our model.
X_test	Contains the remaining percent of our features which will be used to test our predictive model after training. This provides an unbiased evaluation of model performance.
Y_train	Contains the target variable values corresponding to X_train . It is used during model training to learn the relationship between features and the target.
Y_test	Contains the target values corresponding to X_test and is used to evaluate the model's accuracy on data it hasn't seen during training.

Table 3.3: Output Variables from **train_test_split**

3.7.8 Data Scaling

When features are on different scales (different prices), some features may dominate others. Data Scaling can help us. The code is as follows:

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

In the first line of the code snippet, we initialize the scaler by creating an instance **scaler** of the scaler by the argument **scaler = StandardScaler()**.

```
X_train_scaled = scaler.fit_transform(X_train)
```

We use the **fit_transform** function to both determine and apply scaling for our training data. First, the **fit** step figures out the average (mean) and spread (standard deviation) for each feature (each column in `X_train`). Then, the **transform** step uses these measurements to scale the training data accordingly.

```
X_test_scaled = scaler.transform(X_test)
```

We are adjusting the test dataset. The parameter **transform** helps scale the test data by applying the average and spread(standard deviation) calculated from the training data. This process ensures that both the training and the test data sets have similar magnitudes.

3.7.9 Model Training (Random Forest)

The next portion of our program trains a Random Forest regression model on our scaled training data. The code is as follows:

```
model = RandomForestRegressor(n_estimators=100,  
                              random_state=42)  
model.fit(X_train_scaled, y_train)
```

RandomForestRegressor, as we have mentioned earlier, is a machine learning algorithm. Its parameters are:-

- **n_estimators=100**: This sets the number of decision trees that we can use in our model. The greater this number up to a certain point, the better the performance but at the cost of computational time and space.

- **random_state=42**: It is essentially a seed used which ensures that the random elements of the algorithm (such as how the trees are built and which features are used for splitting) are reproducible. This makes our model reproducible.

```
model.fit(X_train_scaled, y_train)
```

The **fit** argument trains the model using the data-set provided in its parameters. **X_train_scaled** is the data from the **features** data frame (independent variables). **y_train** are the **target** values (dependent variable) corresponding to each row in **X_train_scaled**. The model learns the relationship between the features and these target values.

How does RandomForestRegressor work in our Program?

The RandomForestRegressor works in the following way:-

- The RandomForestRegressor creates/builds a collection of 100 decision trees(equal to the number of **n_estimators**). Each of these trees is generated using a randomly selected set within training data (a process called bootstrap sampling). This makes the trees diverse.
- The algorithm considers only a random set within **features** at each step of the decision trees. This makes the decision trees more different from each other and the overall process more robust.
- When constructing each tree, the model identifies decision rules to divide the data in a way that reduces errors in predicting the target variable. Since each tree is trained on a different set within data and features, they can capture slightly varied patterns.
- While making a prediction, each individual decision tree outputs a prediction (a numerical value). The final prediction is the average of these predictions. This averaging reduces the variance and improves overall accuracy.
- We make sure that these random selections of the features and the target variables is consisting by setting **random_state=42** making the split reproducible.

3.7.10 Predictions

The following code snippet demonstrates the prediction phase in our machine learning workflow. It leverages the trained Random Forest model to make predictions on test. The code is provided below:

```
predictions = model.predict(X_test_scaled)
```

Our input is the **X_test_scaled** data-set. This variable contains the scaled features from the test set. The method **model.predict** is a part of **RandomForestRegressor** and is used to generate predictions for new, unseen data. The method works by passing each sample in **X_test_scaled** down every decision tree in the Random Forest.

- Prediction made by each tree is based on the decision rules it learnt during training.
- Such predictions from all the trees are aggregated and the average is calculated to get the final predicted value for each test instance.

As a result, the variable **predictions** stores the model's forecasted values for the target variable. In this case, it represents the average return predicted five days ahead for each row in the test dataset.

3.7.11 Visualization:-

The final portion of our program includes the graphical representation of the predicted trends superposed on the actual trends. The code is as follows:

```
plt.figure(figsize=(12, 6))
plt.plot(y_test.values, label="Actual Returns
(5-day ahead avg)", marker='o', linestyle='--',
color="blue")
plt.plot(predictions, label="Predicted Returns",
marker='x', linestyle='--', color="red")
plt.xlabel("Test Sample Index")
plt.ylabel("5-Day Average Return")
plt.title("Actual vs. Predicted 5-Day Ahead Average Return")
plt.legend(loc='best')
plt.grid('true')
plt.show()
```

Firstly, we set the figure of size 12 inches in width and 6 inches in height.

```
plt.plot(y_test.values, label="Actual Returns  
(5-day ahead avg)", marker='o', linestyle='--',  
color="blue")
```

This plot the **actual return** using circular markers and blue dashed line. Its label will be in the legend.

```
plt.plot(predictions, label="Predicted Returns",  
marker='x', linestyle='--', color="red")
```

This plots the predictions using the cross markers and a red dashed line. Its label will be in the legend.

```
plt.xlabel("Test Sample Index")  
plt.ylabel("5-Day Average Return")  
plt.title("Actual vs.Predicted 5-Day Ahead Average Return")
```

This deals with the labelling of the graph. On the X-axis the label will be **Test Sample Index**. On the Y-axis it will be **5-day average returns** and the title of the graph will be **Actual vs.Predicted 5-Day Ahead Average Return**.

```
plt.legend(loc='best')  
plt.grid('true')
```

This sets the legend for the graph, that shows which line corresponds to actual or predicted returns, in the best location(on the graph). It also turns on the grid for better readability.

```
plt.show()
```

Finally, this displays the graph.

NOTE: As we have not provided the x-values to plt.plot commands, the index of the array (i.e., range(len(y))) is automatically set as the x-values by Matplotlib. So, in our case, the x-values are implicitly set to 0, 1, 2, ..., up to the length of our data minus one.

Chapter 4

Output, Evaluation of Performance of the Model and Deductions

The most important aspect about science is that it heavily dependent on verification. This distinguishes science from other subjects, such as philosophy. Any theory and research must be backed by its verification. Only then does it have any value in the world of science. The results that we can achieve by means of science have to undergo such verification processes. We can verify our work by evaluating them against the actual data that can be obtained from the physical world. Our project in this regard is no exception. Therefore, now we move on with the final part of our project, which deals with getting the output and evaluating its performance

4.1 Evaluation of Regression model

We can evaluate our regression models using three error calculation formulas. They are terms as **Regression Metrics**. They are as follows:-[29]

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- R^2 (R-squared)

4.1.1 Mean Squared Error (MSE)

MSE measures the average squared difference between actual and predicted values. [29, 26, 30]

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (4.1)$$

where, y_i is the actual value, $\hat{y}_i$ is the predicted value and n is the number of samples. **Lower MSE indicates better model accuracy.** Even small difference between the predicted and actual give large MSE because of the square.

4.1.2 Mean Absolute Error (MAE)

It measures the average absolute difference between actual and predicted values. [29, 26, 30]

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.2)$$

Lower MAE means better accuracy of the model. It is less sensitive to large differences compared to MSE

4.1.3 R^2 (R-squared)

It measures how well the model explains the variance in the actual data. Its value ranges from 0 to 1, while sometimes it is negative as well [29, 26, 30]

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4.3)$$

where, $\bar{y}$ is the mean of the actual values. If R^2 value is equal to 1 it shows perfect prediction. If it is equal to zero then the model just helps predict the mean of actual values while if it's negative then the model is much worse in its predictive capabilities.

In the Program we have used the Regression metrics as follows:-[26]

```
from sklearn.metrics import mean_squared_error,
                                mean_absolute_error, r2_score
mse = mean_squared_error(y_test, predictions)
```

```

mae = mean_absolute_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print(f"\nMean Squared Error (MSE): {mse:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"R-squared (R^2): {r2:.4f}")

```

Their value shows us how well our model is working.

4.2 Output

. We will show many outputs each for some different starting conditions:-

4.2.1 Output for our base program

The output we got is as follows:-

Figure 4.1: Command prompt output

```

Loading data for AAPL from AAPL_data.txt ...
Loading data for MSFT from MSFT_data.txt ...
Loading data for GOOG from GOOG_data.txt ...
Loading data for AMZN from AMZN_data.txt ...

Combined Price Data (first 5 rows):
      date      AAPL      MSFT      GOOG      AMZN
2023-01-03 00:00:00+00:00 123.636383 235.255988 89.271543 85.82
2023-01-04 00:00:00+00:00 124.911516 224.965858 88.286272 85.14
2023-01-05 00:00:00+00:00 123.586876 218.297608 86.355539 83.12
2023-01-06 00:00:00+00:00 128.134146 220.870321 87.738899 86.08
2023-01-09 00:00:00+00:00 128.658070 223.020794 88.375842 87.36

Daily Log Returns (first 5 rows):
      date      AAPL      MSFT      GOOG      AMZN
2023-01-04 00:00:00+00:00 0.010261 -0.044729 -0.011098 -0.007955
2023-01-05 00:00:00+00:00 -0.010661 -0.030086 -0.022112 -0.024012
2023-01-06 00:00:00+00:00 0.036133 0.011716 0.015892 0.034992
2023-01-09 00:00:00+00:00 0.004081 0.009689 0.007233 0.014760
2023-01-10 00:00:00+00:00 0.004446 0.007588 0.004943 0.028327

Features DataFrame (first 5 rows):
      AAPL_vol AAPL_ent AAPL_brow AAPL_shm
... AMZN_vol AMZN_ent AMZN_brow AMZN_shm
date
2023-02-15 00:00:00+00:00 0.014283 2.092785 0.022495 0.056675
... 0.029161 1.629720 0.044241 0.159286
2023-02-16 00:00:00+00:00 0.014619 2.136688 0.022612 0.056675
... 0.029770 1.726414 0.044022 0.159286
2023-02-17 00:00:00+00:00 0.014508 2.099497 0.020828 0.056675
... 0.029418 1.725414 0.043704 0.159286
2023-02-21 00:00:00+00:00 0.014665 2.108516 0.020312 0.063434
... 0.029442 1.769317 0.043668 0.159286
2023-02-22 00:00:00+00:00 0.014668 2.108516 0.021086 0.063434
... 0.029418 1.769317 0.044245 0.159286

[5 rows x 16 columns]

Target (first 5 values):
date
2023-02-15 00:00:00+00:00 -0.008929
2023-02-16 00:00:00+00:00 -0.018640
2023-02-17 00:00:00+00:00 -0.010761
2023-02-21 00:00:00+00:00 -0.007329
2023-02-22 00:00:00+00:00 -0.002154
dtype: float64

```

Figure 4.2: Command prompt output

Figure 4.1: Command prompt output Figure 4.2: Command prompt output

Deductions:-

The output we have received is shown above. The graph shows the predicted returns superposed on the curve of the actual returns. If we look closely at ??, we see that even though our predicted model does not exactly trace

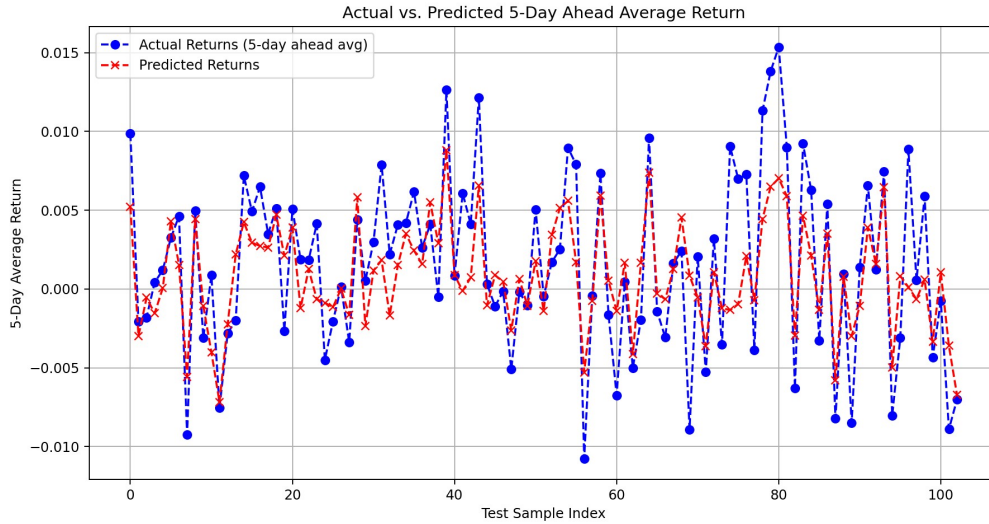


Figure 4.3: Graphical output

the actual returns graph, the crest and trough of the two graphs mostly align and the blue(actual) and red(predicted) lines mostly move in the same direction. This indicates the model is capturing some of the underlying trends or momentum. Even though our predicted graph is missing the amplitude of the actual returns but still is able to capture the general direction of returns pretty well. The regression metrics values are as follows:-

- Mean Squared Error (MSE)= **Manually Computed MSE: 1.3128999501342838e-05**
- Mean Absolute Error (MAE)= **0.0029**
- R^2 (R-squared)= **0.5776**

Thus, the model works quite well in predicting the market trends.

4.2.2 Modified Program/Inputs

Different data-set

We modify our base program so that we have a different data-set. We are taking historical stock data from 10 companies. They are - Apple (AAPL), Microsoft (MSFT), Alphabet (GOOG), Amazon (AMZN), Meta (META),

Tesla (TSLA), Netflix (NFLX), NVIDIA (NVDA), Intel (INTC) and IBM (IBM).

```
Data for AAPL is already cached in AAPL_data.txt.
Data for MSFT is already cached in MSFT_data.txt.
Data for GOOG is already cached in GOOG_data.txt.
Data for AMZN is already cached in AMZN_data.txt.
Fetching data for FB from Tiingo...
Error fetching FB: {'detail': "Error: Ticker 'FB' not found"}
Failed to fetch data for FB.
Fetching data for TSLA from Tiingo...
Cached data for TSLA to TSLA_data.txt.
Fetching data for NFLX from Tiingo...
Cached data for NFLX to NFLX_data.txt.
Fetching data for NVDA from Tiingo...
Cached data for NVDA to NVDA_data.txt.
Fetching data for INTC from Tiingo...
Cached data for INTC to INTC_data.txt.
Fetching data for IBM from Tiingo...
Cached data for IBM to IBM_data.txt.
```

Figure 4.4: Running cache_data.py

Thus, we have obtained the required data after running the modified code for cache_data.py as seen in 4.4. Now on running the main program we get the following output:-

```
Data for AAPL is already cached in AAPL_data.txt.
Data for MSFT is already cached in MSFT_data.txt.
Data for GOOG is already cached in GOOG_data.txt.
Data for AMZN is already cached in AMZN_data.txt.
Fetching data for META from Tiingo...
Cached data for META to META_data.txt.
Data for TSLA is already cached in TSLA_data.txt.
Data for NFLX is already cached in NFLX_data.txt.
Data for NVDA is already cached in NVDA_data.txt.
Data for INTC is already cached in INTC_data.txt.
Data for IBM is already cached in IBM_data.txt.

C:\Users\Hridayesh\Downloads\WPy64-31241\Disser>python disss.py
Loading data for AAPL from AAPL_data.txt ...
Loading data for MSFT from MSFT_data.txt ...
Loading data for GOOG from GOOG_data.txt ...
Loading data for AMZN from AMZN_data.txt ...
Loading data for META from META_data.txt ...
Loading data for TSLA from TSLA_data.txt ...
Loading data for NFLX from NFLX_data.txt ...
Loading data for NVDA from NVDA_data.txt ...
Loading data for INTC from INTC_data.txt ...
Loading data for IBM from IBM_data.txt ...

Combined Price Data (first 5 rows):
...
date
...
2023-01-03 00:00:00+00:00 123.636303 235.255908 89.271543 85.82 ...
```

Figure 4.5: Command prompt output

```
... 294.95 14.302799 25.759480 129.248359
2023-01-04 00:00:00+00:00 124.911516 229.965058 88.286272 85.14 ...
... 309.41 14.726029 26.670987 130.207107
2023-01-05 00:00:00+00:00 123.586876 218.297608 86.355539 83.12 ...
... 309.70 14.252841 26.559344 128.846598
2023-01-06 00:00:00+00:00 128.134146 220.870321 87.738899 86.08 ...
... 315.55 14.846335 27.686864 131.211510
2023-01-09 00:00:00+00:00 128.658070 223.020794 88.375842 87.36 ...
... 315.17 15.614680 28.245805 131.074546

[5 rows x 10 columns]

Daily Log Returns (first 5 rows):
...
NFLX NVDA INTC AAPL MSFT GOOG AMZN ...
IBM
date
2023-01-04 00:00:00+00:00 0.010261 -0.0044729 -0.011098 -0.007955 ...
0.047861 0.029867 0.034924 0.007390
2023-01-05 00:00:00+00:00 -0.010661 -0.030086 -0.022112 -0.024012 ...
0.000937 -0.033366 -0.004345 -0.010504
2023-01-06 00:00:00+00:00 0.036133 0.011716 0.015892 0.034992 ...
0.013713 0.040797 0.041576 0.018188
2023-01-09 00:00:00+00:00 0.004081 0.009689 0.007233 0.014760 ...
-0.001205 0.050458 0.019987 -0.001044
2023-01-10 00:00:00+00:00 0.004446 0.007588 0.004943 0.028327 ...
0.038498 0.017821 0.004426 0.008670

[5 rows x 10 columns]

Features DataFrame (first 5 rows):
```

Figure 4.6: Command prompt output

Deductions:-

We see that, similar to the earlier graph, we have received a graph with the predicted (red) returns plot superposed on the actual (blue) returns plot. Also, similar to the above case, even though our predicted graph is missing

```

.   IBM_vol   IBM_ent   AAPL_vol AAPL_ent   AAPL_brow AAPL_shm ..
date
2023-02-15 00:00:00+00:00 0.014283 2.092785 0.022495 0.056675 ..
.   0.012363 1.626541 0.017575 0.064052
2023-02-16 00:00:00+00:00 0.014619 2.136688 0.022612 0.056675 ..
.   0.012379 1.637868 0.017431 0.064052
2023-02-17 00:00:00+00:00 0.014508 2.099497 0.020828 0.056675 ..
.   0.012266 1.586785 0.016699 0.064052
2023-02-21 00:00:00+00:00 0.014665 2.108516 0.020312 0.063434 ..
.   0.012444 1.744793 0.016950 0.059601
2023-02-22 00:00:00+00:00 0.014668 2.108516 0.021086 0.063434 ..
.   0.012454 1.744793 0.017239 0.059601

[5 rows x 40 columns]

Target (first 5 values):
date
2023-02-15 00:00:00+00:00 -0.011919
2023-02-16 00:00:00+00:00 -0.010667
2023-02-17 00:00:00+00:00 -0.008903
2023-02-21 00:00:00+00:00 -0.005738
2023-02-22 00:00:00+00:00 0.000445
dtype: float64

Mean Squared Error (MSE): 0.0000
Mean Absolute Error (MAE): 0.0035
R-squared (R^2): 0.5534

```

Figure 4.7: Running cache_data.py

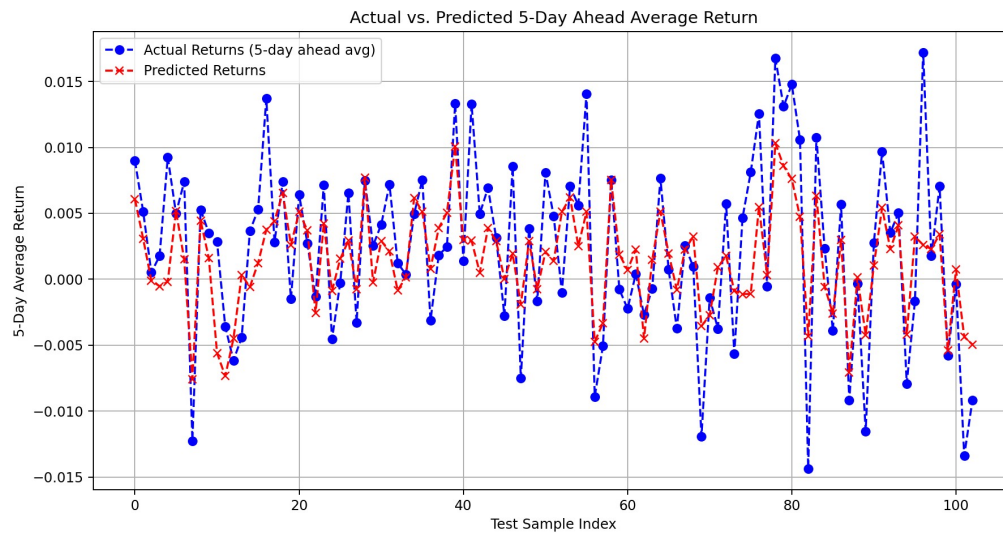


Figure 4.8: Graphical output

the amplitude of the actual returns but still is able to capture the general direction of returns. However we see that the predictions is not as good compared to the previous case with lesser number of data-sets. This can also be seen from the regression metric calculation:

- Mean Squared Error (MSE)= Manually Computed MSE:
2.0094481645945618e-05

- Mean Absolute Error (MAE)= **0.0035**
- R^2 (R-squared)= **0.5534**

The greater values of MSE and MAE, and the lesser Value of R^2 show us that the prediction is not as accurate as the earlier results. Compared to the previous, not as many crests and troughs are matching in this case. Nonetheless, the regressor is doing a great job at predicting the market.

Increasing Prediction Horizon (10-Day Ahead Instead of 5-Day)

Now we modify our programming model to predict 10-day ahead average return instead of 5-day ahead average returns. The output we get is as follows:

```
Loading data for AAPL from AAPL_data.txt ...
Loading data for MSFT from MSFT_data.txt ...
Loading data for GOOG from GOOG_data.txt ...
Loading data for AMZN from AMZN_data.txt ...

Combined Price Data (first 5 rows):
      AAPL      MSFT      GOOG      AMZN
date
2023-01-03 00:00:00+00:00  123.636303  235.255908  89.271543  85.82
2023-01-04 00:00:00+00:00  124.911516  224.965058  88.286272  85.14
2023-01-05 00:00:00+00:00  123.586876  218.297608  86.355539  83.12
2023-01-06 00:00:00+00:00  128.134146  220.870321  87.738899  86.08
2023-01-09 00:00:00+00:00  128.658070  223.020794  88.375842  87.36

Daily Log Returns (first 5 rows):
      AAPL      MSFT      GOOG      AMZN
date
2023-01-04 00:00:00+00:00  0.010261 -0.044729 -0.011098 -0.007955
2023-01-05 00:00:00+00:00 -0.010661 -0.030086 -0.022112 -0.024012
2023-01-06 00:00:00+00:00  0.036133  0.011716  0.015892  0.034992
2023-01-09 00:00:00+00:00  0.004081  0.009689  0.007233  0.014760
2023-01-10 00:00:00+00:00  0.004446  0.007588  0.004943  0.028327

Features DataFrame (first 5 rows):
      AAPL_vol AAPL_ent ... AMZN_brow AMZN_sh
m
date
2023-02-15 00:00:00+00:00  0.014283  2.092785 ... 0.044241  0.15928
6
```

Figure 4.9: Command prompt output

```
      AAPL_vol AAPL_ent ... AMZN_brow AMZN_sh
m
date
2023-02-15 00:00:00+00:00  0.014283  2.092785 ... 0.044241  0.15928
6
2023-02-16 00:00:00+00:00  0.014619  2.136688 ... 0.044922  0.15928
6
2023-02-17 00:00:00+00:00  0.014508  2.099497 ... 0.043704  0.15928
6
2023-02-21 00:00:00+00:00  0.014665  2.108516 ... 0.043668  0.15928
6
2023-02-22 00:00:00+00:00  0.014668  2.108516 ... 0.044245  0.15928
6

[5 rows x 16 columns]

Target (first 5 values):
date
2023-02-15 00:00:00+00:00  -0.006919
2023-02-16 00:00:00+00:00  -0.006917
2023-02-17 00:00:00+00:00  -0.002408
2023-02-21 00:00:00+00:00  -0.000558
2023-02-22 00:00:00+00:00  0.000961
dtype: float64

Mean Squared Error (MSE): 0.0000
Mean Absolute Error (MAE): 0.0015
R-squared (R^2): 0.7632
```

Figure 4.10: Command prompt output

Deductions:-

We see that even for the case of 10-day ahead prediction, the model works exceptionally well. We have received a graph with the predicted (red) returns plot superposed on the actual (blue) returns plot. When we compare the present output graph with the 1st output graph we see that the amplitude of the predicted returns line in this case is greater than the actual return amplitude in most regions compared to the 1st output for 5-day ahead. There are

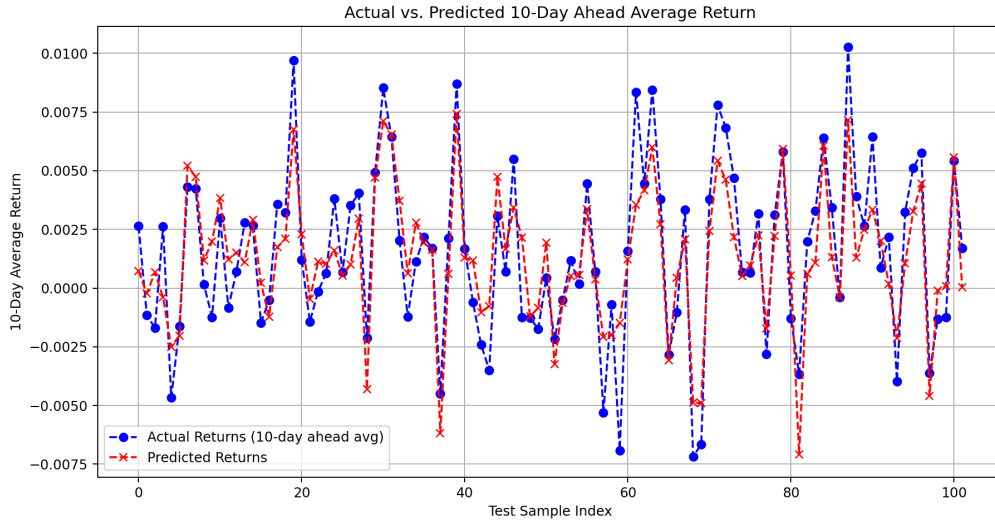


Figure 4.11: Graphical output

more visible deviations between the predicted and actual returns, suggesting higher forecasting errors over a longer prediction horizon. The 10-day return prediction has higher fluctuations, with more extreme peaks and troughs. The regression metrics are as follows:

- Mean Squared Error (MSE)= **Manually Computed MSE: 3.1668824090382797e-06**
- Mean Absolute Error (MAE)= **0.0015**
- R^2 (R-squared)= **0.7632**

The 10-day ahead model performs better than the 5-day model in terms of MSE, MAE, and R^2 likely because longer-term averages smooth out short-term market noise, making predictions easier. The 5-day model is more sensitive to volatility, leading to higher errors and lower R^2 . Additionally, the 10-day model may avoid overfitting to short-term fluctuations, making it more stable and reliable for capturing overall trends. This suggests that while the 5-day model reacts more to short-term changes, the 10-day model provides a more consistent and accurate forecast.

Setting different date range

Now, we are setting the date range from 10th of March, 2019 to 10th of March, 2025 and predict 10-day ahead average return . Output is as follows:

```
Loading data for AAPL from AAPL_data.txt ...
Loading data for MSFT from MSFT_data.txt ...
Loading data for GOOG from GOOG_data.txt ...
Loading data for AMZN from AMZN_data.txt ...

Combined Price Data (first 5 rows):
      date      AAPL      MSFT      GOOG      AMZN
2019-03-11 00:00:00+00:00  42.852968  106.577358  58.507196  83.5310
2019-03-12 00:00:00+00:00  43.334435  107.323579  59.375031  83.6550
2019-03-13 00:00:00+00:00  43.526064  108.154012  59.381002  84.5405
2019-03-14 00:00:00+00:00  44.009926  108.239825  58.994358  84.3110
2019-03-15 00:00:00+00:00  44.582417  109.486675  58.940118  85.6100

Daily Log Returns (first 5 rows):
      date      AAPL      MSFT      GOOG      AMZN
2019-03-12 00:00:00+00:00  0.011173  0.006977  0.014724  0.001403
2019-03-13 00:00:00+00:00  0.004412  0.007715  0.000101  0.010530
2019-03-14 00:00:00+00:00  0.011055  0.000786 -0.006533 -0.002718
2019-03-15 00:00:00+00:00  0.012924  0.011453 -0.000920  0.015383
2019-03-18 00:00:00+00:00  0.010157  0.014220 -0.000169  0.017247

Features DataFrame (first 5 rows):
      AAPL_vol AAPL_ent AAPL_brow AAPL_shm
.. AMZN_vol AMZN_ent AMZN_brow AMZN_shm
date
..
```

Figure 4.12: Command prompt out-
put

```
date
..
2019-04-23 00:00:00+00:00  0.010608  1.927695  0.015457  0.057094
.. 0.010076  1.827983  0.014287  0.050314
2019-04-24 00:00:00+00:00  0.010605  1.922113  0.015684  0.057094
.. 0.010477  1.923677  0.015280  0.050314
2019-04-25 00:00:00+00:00  0.010883  1.958934  0.015685  0.057094
.. 0.010434  1.911384  0.015257  0.050314
2019-04-26 00:00:00+00:00  0.010916  1.967356  0.015705  0.057094
.. 0.011023  1.847135  0.015578  0.055524
2019-04-29 00:00:00+00:00  0.010780  1.941862  0.015749  0.057094
.. 0.011019  1.875904  0.016613  0.055524

[5 rows x 16 columns]

Target (first 5 values):
date
2019-04-23 00:00:00+00:00  0.000954
2019-04-24 00:00:00+00:00 -0.002438
2019-04-25 00:00:00+00:00 -0.002063
2019-04-26 00:00:00+00:00 -0.003394
2019-04-29 00:00:00+00:00 -0.004332
dtype: float64

Mean Squared Error (MSE): 0.0000
Mean Absolute Error (MAE): 0.0019
R-squared (R^2): 0.7208
```

Figure 4.13: Command prompt out-
put

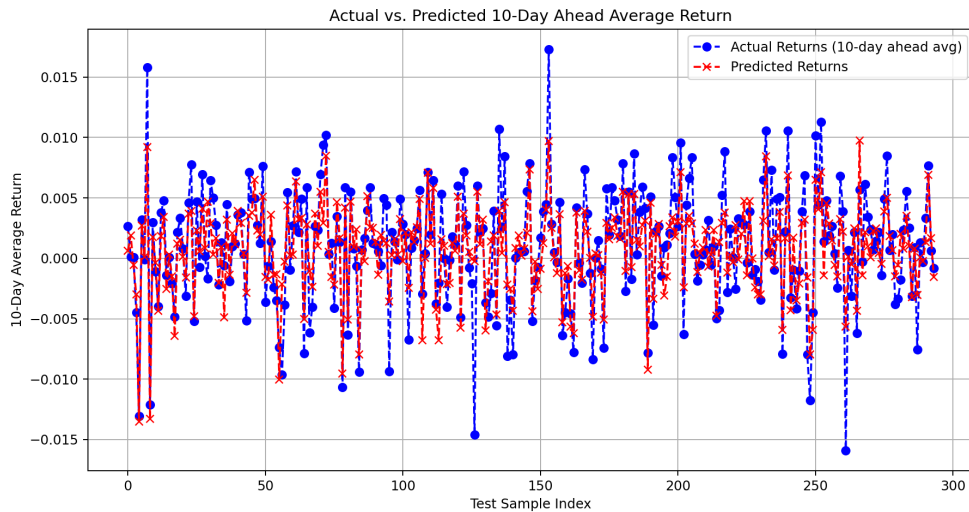


Figure 4.14: Graphical output

Deductions

Here also we see that even if the predictive model (red line) is missing the amplitudes it is still able to some extent plot the crests and troughs of the actual returns line (blue line). This indicates the model is capturing some of the underlying trends or momentum. Even though our predicted graph is missing the amplitude of the actual returns but still is able to capture the general direction of returns pretty well. The regression metrics values are as follows:-

- Mean Squared Error (MSE)= **Manually Computed MSE: 6.729135726931776e-06**
- Mean Absolute Error (MAE)= **0.0019**
- R^2 (R-squared)= **0.7208**

The 4 different program outputs are summarized below:

Table 4.1: Performance Metrics for Different Program Variants

Description	MSE	MAE	R^2
Base Program	1.3129×10^{-5}	0.0029	0.5776
Different data-set (data of 10 companies)	2.0094×10^{-5}	0.0035	0.5534
Increasing Prediction Horizon (10-Day Ahead Instead of 5-Day)	3.1669×10^{-6}	0.0015	0.7632
Setting different date range (10th March 2019 to 10th March 2025)	6.7291×10^{-6}	0.0019	0.7208

Deductions From the Table

The best outcome is when **Increasing the prediction horizon to 10 days (Scenario 3)** which yields the **lowest error** (MSE, MAE) and the **highest R^2** , suggesting that in this dataset, extending the forecast window

may capture medium-term trends more effectively than a shorter horizon. Different date range (2019–2025) also produces strong results, indicating that this time period might be more favourable or more predictable for the model. The 10-companies scenario introduces more noise in the prediction as the base program is better. **None of the scenarios yield R^2 near 1 (perfect), but all are above 0, meaning each model does better than simply predicting the mean.**

- When the red line (predicted line) stays above zero, our model anticipates gains and signals to hold or increase our position. If it dips below zero, the model expects losses in which case we can consider reducing our stocks then to protect against a potential downturn.
- we notice that the red line moves in the same direction as the blue line which shows that our model is capturing the trend. This gives us more confidence to act on its signals. however, if in case red line frequently strays far from the blue line, the model might not be reliable.
- When the amplitude of the blue line is greater than the red line then our model is not predictive volatility well. However, if it is the other way round, we will end up taking more risk.
- Finally, if the red line (predicted) shows clear, sustained positive trends then its a signal to increase our position. However, if it predicts a dip we can trim positions or shift our capital to other opportunities.

Conclusion

We finally have achieved our desired result. Using the output and generated graphs, we can come up with investment strategies. These strategies will help maximize our profits and returns, and minimize our risk and losses. Thus, we can optimize our portfolios in the best possible way due to our predictive advantage unlike no other.

This is a very simple project that we have built in order to optimize portfolios. Much more advanced versions of stock optimization programs that use machine learning can be created by factoring in additional conditions and relations. We can also implement an adaptive trading strategy in which portfolios adjust over time based on predicted returns and volatility. Another possible extension is using Reinforcement Learning models which train based on the trial and error method. A modified program can be made which works on real time stock data and tries to predict future returns based on past trends and physics-inspired concepts. Attempt in integrating external factors such as news, market sentiments, calamities, and so on can also be made to make our model more precise and ready for real world scenarios.

The applications of machine learning, however, does not stop only in the financial sector but stretches further into every other field imaginable. Key applications include those in the medical sector, artificial intelligence sector, natural language processing, automated vehicles, robotics, and so on. As mentioned earlier, this world thrives on data and the power to store, analyse and use data can help one conquer the world. With the help of machine learning one can reach heights that seem impossible for humans. for us the sky is the limit, and much beyond.

References

- [1] Xenit. *Machine Learning Diagram*. Accessed: 2025-04-03. Accessed: 2025-04-03. URL: <https://xenit.eu/wp-content/uploads/2020/01/machine-learning.jpg>.
- [2] Vecteezy. *Machine Learning Types: Supervised vs Unsupervised vs Reinforcement in a Glance*. Accessed: 2025-04-03. Accessed: 2025-04-03. URL: https://static.vecteezy.com/system/resources/previews/034/796/117/large_2x/machine-learning-types-supervised-vs-unsupervised-vs-reinforcement-in-a-glance-editable-stroke-and-colors-vector.jpg.
- [3] IBM. *What is Supervised Learning?* <https://www.ibm.com/topics/supervised-learning>. Accessed: 2025-04-06. 2023.
- [4] Scikit-Learn Developers. *Scikit-Learn: Machine Learning in Python*. <https://scikit-learn.org/stable/index.html>. Comprehensive documentation for the scikit-learn machine learning library.
- [5] Wikipedia contributors. *Scikit-learn — Wikipedia*. Accessed: 2025-04-05. 2023. URL: <https://en.wikipedia.org/wiki/Scikit-learn>.
- [6] Freepik. *Portfolio Optimization Growth Line Icon*. Accessed: 2025-04-03. Accessed: 2025-04-03. URL: https://img.freepik.com/premium-vector/portfolio-optimization-growth-line-icon_116137-3242.jpg?w=1060.
- [7] Tiingo Inc. *Tiingo: Financial Data Platform*. Accessed: 2025-04-03. 2023. URL: <https://www.tiingo.com/>.
- [8] F. Reif. *Fundamentals of Statistical and Thermal Physics*. A classic text on statistical mechanics; includes foundational treatments of Boltzmann entropy. New York, NY, USA: McGraw-Hill, 1965.

- [9] S. J. Blundell and K. M. Blundell. *Concepts in Thermal Physics*. 2nd. Includes clear discussions on entropy and thermodynamics. Oxford, UK: Oxford University Press, 2010.
- [10] C. E. Shannon. “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27.3 (1948). Seminal paper defining the concept of information entropy., pp. 379–423.
- [11] Lucretius. *On the Nature of Things*. Ed. and trans. by Martin Ferguson Smith. Book II, verses 113–140. Indianapolis: Hackett Publishing Company, 2001.
- [12] Robert Brown. “A Brief Account of Microscopical Observations on the Particles contained in the Pollen of Plants, and on the General Existence of Active Molecules in Organic and Inorganic Bodies”. In: *Philosophical Magazine* 4 (1828), pp. 161–173.
- [13] Wikimedia Commons. *Brownian Motion Animation*. Accessed: 2025-04-03. URL: https://upload.wikimedia.org/wikipedia/commons/c/c2/Brownian_motion_large.gif.
- [14] A. Einstein. “On the Movement of Small Particles Suspended in Stationary Liquids Required by the Molecular-Kinetic Theory of Heat”. In: *Annalen der Physik* 322.8 (1905). Foundational work on Brownian motion., pp. 549–560.
- [15] Ioannis Karatzas and Steven Shreve. *Brownian Motion and Stochastic Calculus*. 2nd. Vol. 113. Graduate Texts in Mathematics. Springer, 1991.
- [16] F. Black and M. Scholes. “The Pricing of Options and Corporate Liabilities”. In: *Journal of Political Economy* 81.3 (1973). Seminal work on option pricing, applying stochastic calculus in finance., pp. 637–654.
- [17] Frederick Reif. *Fundamentals of Statistical and Thermal Physics*. McGraw-Hill, 2009.
- [18] IG Group. *Oil - US Crude*. Accessed: 2025-04-03. 2019. URL: https://a.c-dn.net/c/content/dam/publicsites/igcom/uk/images/ContentImage/Oil%20-%20US%20Crude_20190108_12.09.png.
- [19] B. K. Chakrabarti et al. *Econophysics of Income and Wealth Distributions*. An important Indian contribution to econophysics, discussing statistical mechanics approaches to financial systems. Cambridge, UK: Cambridge University Press, 2013.

- [20] H. Markowitz. “Portfolio Selection”. In: *Journal of Finance* 7.1 (1952). Foundation of modern portfolio theory and risk management., pp. 77–91.
- [21] MME Revise. *Simple Harmonic Motion Diagram*. Accessed: 2025-04-03. 2025. URL: <https://mmerevise.co.uk/app/uploads/2022/11/Picture60.png.webp>.
- [22] D. Halliday, R. Resnick, and J. Walker. *Fundamentals of Physics*. 9th. Covers basic physics principles, including Hooke’s law and SHM. Hoboken, NJ, USA: Wiley, 2010.
- [23] W. McKinney. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. 2nd. Essential resource for data analysis in Python. Sebastopol, CA, USA: O’Reilly Media, 2017.
- [24] C. M. Bishop. *Pattern Recognition and Machine Learning*. Comprehensive overview of machine learning fundamentals. New York, NY, USA: Springer, 2006.
- [25] Keboola. *Random Forest Regression: An Introduction*. Accessed: 2025-04-03. 2025. URL: <https://www.keboola.com/blog/random-forest-regression>.
- [26] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011). Describes the design and implementation of scikit-learn, including utilities such as `train_test_split`., pp. 2825–2830.
- [27] The Pandas Development Team. *Pandas: Powerful Python Data Analysis Toolkit*. Accessed: 2025-04-03. 2023. URL: <https://pandas.pydata.org/>.
- [28] Pandas Documentation. *Rolling Window Statistics in Pandas*. 2023. URL: https://pandas.pydata.org/pandas-docs/stable/user_guide/window.html.
- [29] Douglas C. Montgomery and George C. Runger. *Applied Statistics and Probability for Engineers*. John Wiley & Sons, 2021.
- [30] IBM. *What Is Linear Regression?* <https://www.ibm.com/think/topics/linear-regression>. Accessed: 2025-04-06. 2023.